Table of Contents

		Page Numbers
1. Introduction		
1.1	Purpose	3
1.2	Intended Customers/Users	3
1.3	Product Scope	3
1.4	Outline of the document	4-5
2. Overall Description		
2.1	Product Perspective	6
2.2	Product Functions	6
2.3	User Classes	6-7
2.4	Design and Implementation Constraints	8-9
3. Diagrams		
3.1	UML Diagrams	10-41
3.2	ER Diagrams	42
3.3	DFD Diagrams	43
4. External Interface Requirements/Screenshots		
4.1	User Interfaces	44
4.2	Hardware Interfaces	44
4.3	Software Interfaces	44-45
4.4	Communication Interfaces	44-45
5. Other Non-Functional Requirements		
5.1	Performance Requirements	46
5.2	Safety Requirements	46
5.3	Security Requirements	46-47
5.4	Software Quality Attributes	47
5.5	Business Policies/Rules (if any)	47-48
6. Testing		
6.1	Block-Box Testing	50-63
6.2	White-Box Testing	50-63
7. Conclusion		64
8.References .9 Acknowledgements		64-65

Introduction:-

- Road accidents are increasing day by day
- Many accidents are not reported immediately
- Delay in emergency help can cause serious loss of life
- Smartphones can help using sensors, GPS, and maps
- This software focuses on road safety and quick alerts
- The system helps users, families, and hospitals
- SRS document explains how the software is planned and designed

1.1Purpose:-

- To improve road safety using a mobile application
- To automatically detect accidents
- To alert the user immediately after an accident
- To send accident location to emergency contacts
- To notify nearby hospitals using GPS location
- To alert users about road construction and blocked routes
- To reduce delay in emergency response
- This SRS document defines requirements and system behavior

1.2Intended Customers/Users:-

- Vehicle drivers and riders using the application
- Users who want safety alerts during travel
- Family members or friends as emergency contacts
- Nearby hospitals receiving accident alerts
- Ambulance or emergency services
- System administrator for managing data

1.3 Product Scope:-

- Automatic accident detection using mobile sensors
- Real-time location tracking using GPS
- Emergency alert notifications with location details
- Alerting nearby hospitals during accidents
- Road construction and blockage notifications using maps
- Manual SOS button for emergencies
- Safe and easy-to-use mobile interface

➢ 1.4 Outline of the document:-

Chapter 1 – Introduction

Introduces the project background, explains the purpose of developing the system, identifies the intended users, defines the product scope, and outlines the structure of the document.

Chapter 2 – Overall Description

Provides a clear, high-level overview of the system, including the working environment, main system functions, different user classes, and the design and implementation constraints.

Chapter 3 – Diagrams

Presents system design diagrams such as Use Case, ER, and DFD diagrams to visually explain how the system works, along with assumptions and dependencies.

Chapter 4 – External Interface Requirements / Screenshots

Describes the user interface of the application, hardware and software interfaces used, communication requirements, and includes screenshots of the system.

Chapter 5 – Other Non-Functional Requirements

Explains system performance, security, safety, reliability, quality attributes, and business rules that the system must follow.

Chapter 6 – Testing

Describes the testing approaches used in the project, including black-box testing and white-box testing, along with test cases and results.

Chapter 7 – Conclusion

Summarizes the entire project and highlights the outcomes, usefulness, and benefits of the system.

Chapter 8 – References

Lists the books, research papers, websites, and other resources referred to during the development of the project.

Chapter 9 – Acknowledgements

Expresses sincere thanks to the project guide, faculty members, institution, and others who supported and guided the project.

2. Overall Description:-

- The system is designed to improve road safety using a mobile application
- It works continuously in the background while the user is travelling
- The system collects data from mobile sensors and GPS
- It processes accident detection, alerts, and route safety notifications
- The system helps users, emergency contacts, and hospitals

➢ 2.1 Product Perspective:-

- This system is a standalone Android application
- It works independently on the user's smartphone
- It is not a part of any existing government or traffic system
- The system interacts with mobile sensors (accelerometer, gyroscope)
- It uses GPS services for location tracking
- It interacts with map services to identify road construction or blockages
- It uses SMS or notification services to send alerts

➢ 2.2 Product Functions:-

- Detects road accidents automatically using mobile sensors
- Alerts the user immediately after accident detection
- Sends emergency alerts to registered contacts
- Sends accident details to nearby hospitals based on location
- Tracks real-time user location using GPS
- Notifies users about road construction and blocked routes
- Provides a manual SOS option for emergency situations
- Stores emergency contact details securely

➢ 2.3 User Classes:-

1.Normal User (Vehicle User / Rider)

Access Levels:

- Can register and login into the application
- Can enable accident detection and location tracking
- Can add and manage emergency contacts
- Can receive alerts about accidents and unsafe routes
- Can receive notifications about road construction or blockages
- Can use manual SOS option during emergency

2. Emergency Contact (Family / Friend)

Access Levels:

- Receives emergency alerts when an accident is detected
- ReceiveS user's location details through SMS or notification
- Can take immediate action to help the user
- Does not have access to system configuration

3. Hospital / Emergency Service

Access Levels:

- Receives accident alert notifications
- Receives user's accident location details
- Can use the information for faster medical response
- Access is limited to emergency information only

4. Administrator

Access Levels:

- Can login with administrator privileges
- Can manage user and emergency contact data
- Can manage hospital or emergency service information
- Can monitor system usage and alert logs
- Can update system-related configurations

5. System Maintainer

Access Levels:

- Maintains overall system performance
- Manages data security and backups
- Handles system updates and bug fixes
- Ensures smooth functioning of sensors, GPS, and services

➢ 2.4 Design and Implementation Constraints:-

Hardware Limits:-

- Requires an Android smartphone
- Device must support accelerometer and GPS sensors
- Performance depends on mobile processor and RAM
- Battery consumption may increase due to background services

Software Requirements:-

- Android operating system is mandatory
- Requires Google Maps or similar map service support
- SMS and notification services must be available
- Application depends on device sensor APIs

Regulatory Rules:-

- User permission is required to access location and sensors
- Emergency alert messages must follow local communication rules
- User data must be handled according to privacy policies
- No misuse of emergency services is allowed

Network Dependency:-

- Internet connection is required for map services
- SMS service is required for sending emergency alerts
- Network issues may delay notifications
- Offline functionality is limited

Database Size:-

- Stores limited user and contact information
- Local database size is restricted by device storage
- Large data storage is not supported
- Old data may need periodic cleanup
- Browser-Specific Issues
- Not applicable as the system is a mobile application
- Works on Android application environment only
- No dependency on web browsers

Security Constraints:-

- User personal data must be stored securely
- Location information must be protected from unauthorized access
- Secure access to emergency contact details is required
- Application should prevent unauthorized usage

3.Diagrams

3.1 UML Diagrams

a) Use Case Diagrams:-

Module 1:- User Registation & Profile Management

- **Description:-**

  The User Registration & Permissions Management module allows users to create an account and securely access the application. This module ensures that the user grants necessary permissions such as location, sensors, and notifications, which are essential for accident detection and emergency alert functionality. It also allows users to add emergency contacts who will be notified during emergencies.

- **Actors:-**

  - **Primary Actor**:- Normal User

- **Use Cases**:-

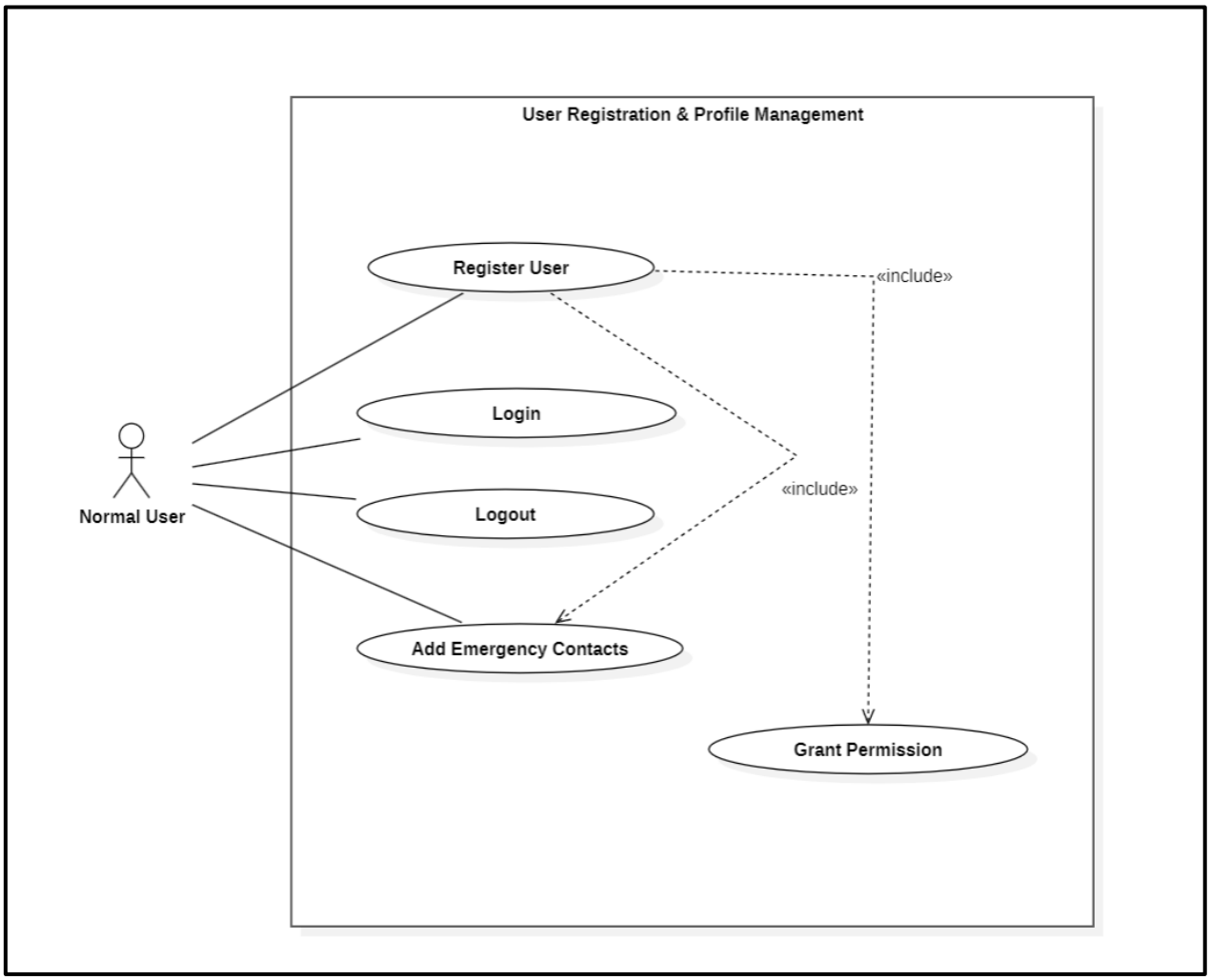
  - Register User
  - Login
  - Logout
  - Add Emergency Contact
  - Grant permission

- **Relationships**:-

  - User must register before logging into the application.
  - Login is required to add emergency contacts.
  - Grant Permissions is required to allow access to location, sensors, and notifications within the application.

➢ Use case Diagram Description

- The use case diagram shows the interactions between the actors and the system, representing the functional requirements of the application.
- It provides a clear visual understanding of system behavior, actor responsibilities, and relationships between different use cases



Module 2:- Accident Detection

- **Description:-**

  The Accident Detection module is responsible for automatically identifying road accidents using mobile sensor data. This module continuously monitors sensor values to detect sudden impacts or abnormal movements. When an accident is detected, the system triggers an alert alarm to confirm the situation and allows the user to cancel false alarms if no accident has occurred.

- **Actors:-**

  - **Primary Actor:** System
  - **Secondary Actor:** Normal user

- **Use Cases:**

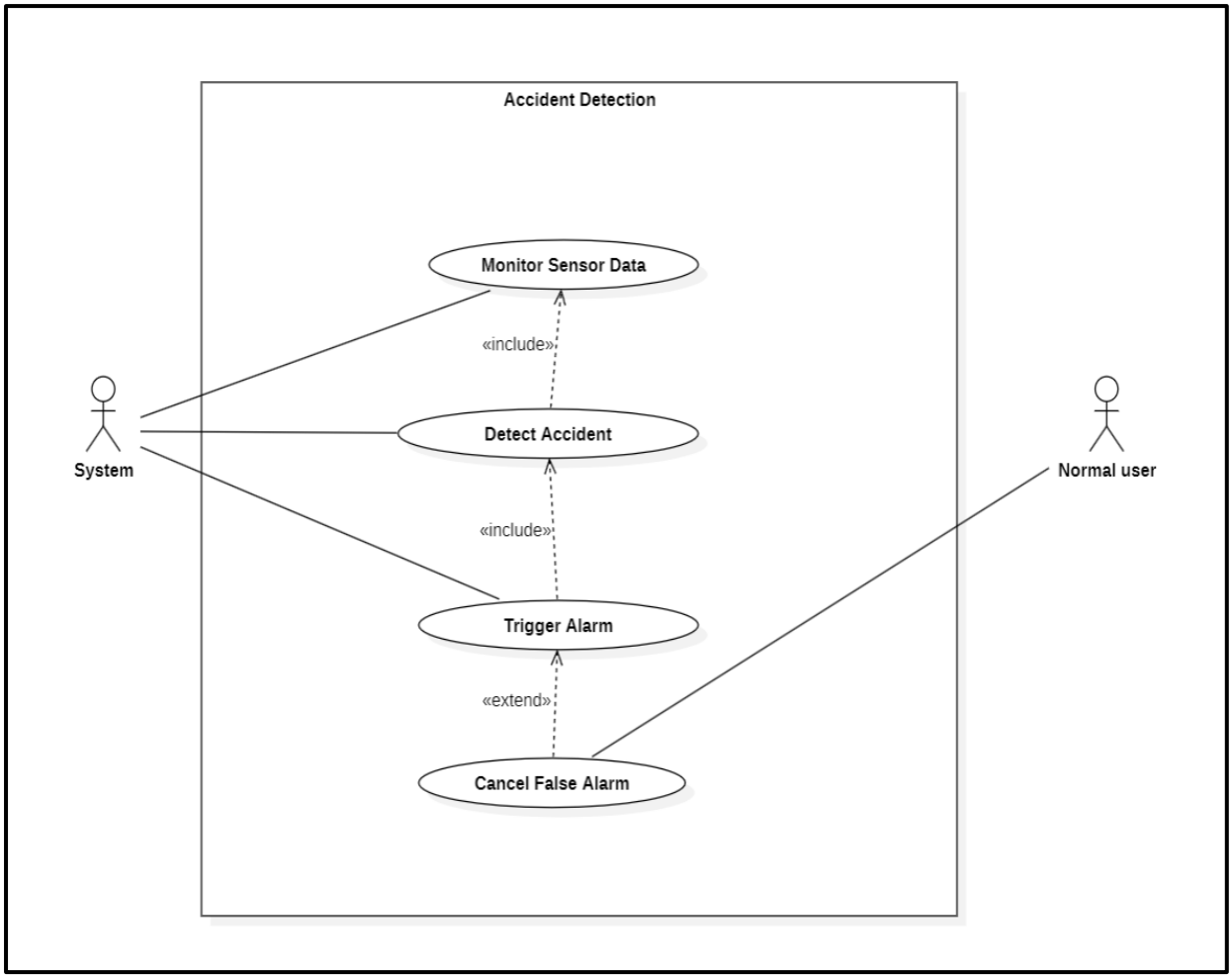
  - Monitor Sensor Data
  - Detect Accident
  - Trigger Alarm
  - Cancel False Alarm

- **Relationships:-**

  - Detect Accident includes Monitor Sensor Data to continuously analyze sensor information.
  - Detect Accident includes Trigger Alarm to alert the user immediately after detection.
  - Cancel False Alarm is an optional action that allows the user to stop the alert when no accident has occurred

- **Use Case Diagram Description:-**

  - The use case diagram represents the automatic accident detection process using mobile sensor data and the triggering of an alert alarm by the system.
  - It shows how the user can interact with the system to cancel false alarms when no accident has occurred.


Accident Detection
Monitor Sensor Data
«include»
Detect Accident
«include»
Trigger Alarm
«extend»
Cancel False Alarm
System
Normal user

Module 3:- Emergency Alert & Notification

- **Description:-**

  The Emergency Alert & Notification module is responsible for sending immediate alerts when an accident is confirmed. It ensures that emergency contacts and nearby hospitals receive timely information along with the accident location. This module helps in reducing emergency response time and provides quick communication during critical situations.

- **Actors:-**

  - **Primary Actor:-** System
  - **Secondary Actors:**- Emergency Contact, Emergency Service

- **Use Cases:-**

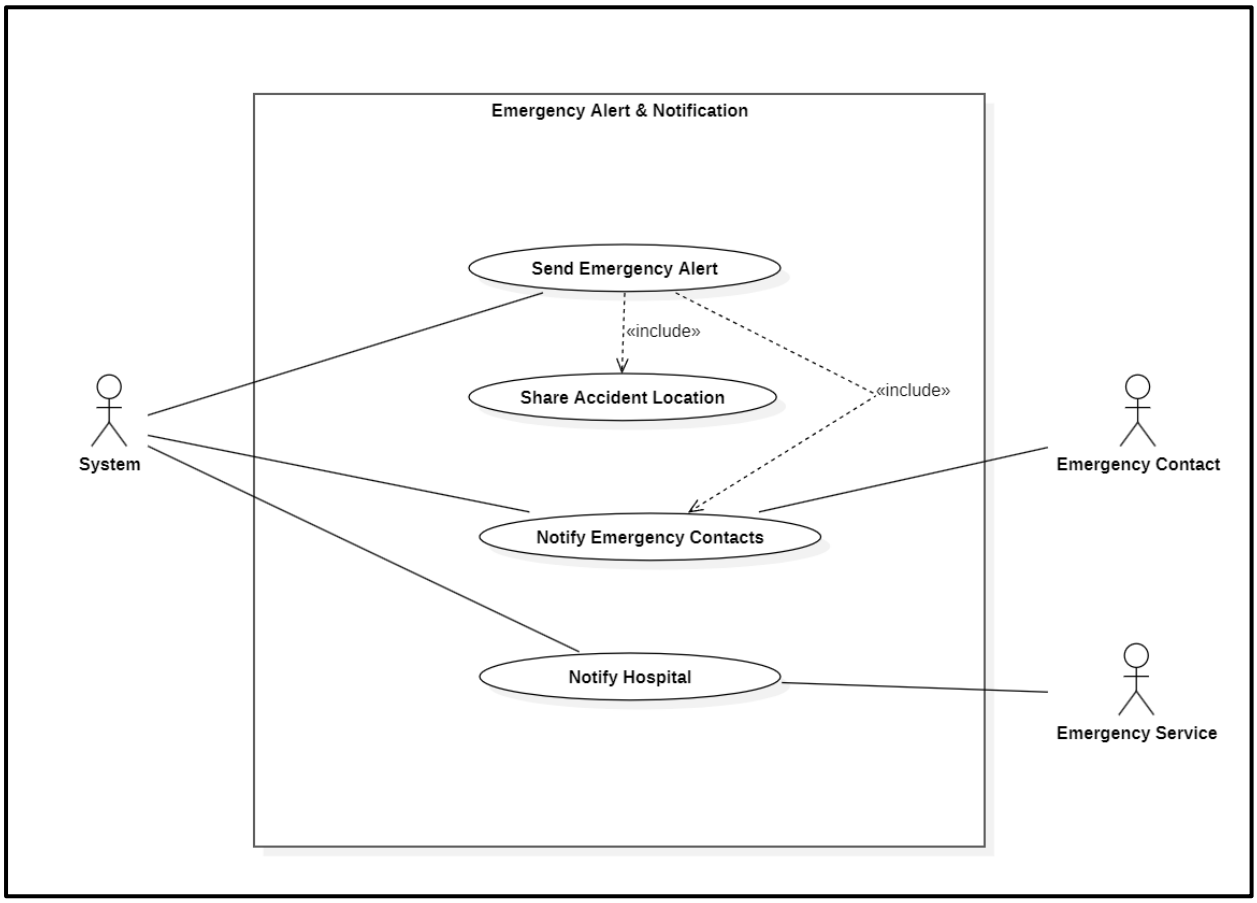
  - Send Emergency Alert
  - Share Accident Location
  - Notify Emergency Contacts
  - Notify Hospital / Emergency Service

- **Relationships:-**

  - Send Emergency Alert includes Share Accident Location to provide accurate location details.
  - Send Emergency Alert includes Notify Emergency Contacts to inform family or friends immediately.
  - Notify Hospital / Emergency Service is performed to ensure medical assistance when required

- **Use Case Diagram Description:-**

  - The use case diagram shows how the system sends emergency alerts with location details after an accident is detected.
  - It illustrates the interaction between the system, emergency contacts, and hospitals for effective emergency response.


Emergency Alert & Notification
Send Emergency Alert
«include»
Share Accident Location
«include»
System
Emergency Contact
Notify Emergency Contacts
Notify Hospital
Emergency Service

Module 4:- Location Tracking

- **Description :-**

  The Location Tracking module is responsible for tracking the user's geographical position using GPS services. It allows the system to obtain real-time location information, which is essential for accident detection, emergency alerts, and safety services. This module ensures accurate and continuous location monitoring while the user is travelling.

- **Actors :-**

  - **Primary Actor**: System
  - **Secondary Actor**: Normal User

- **Use Cases :-**

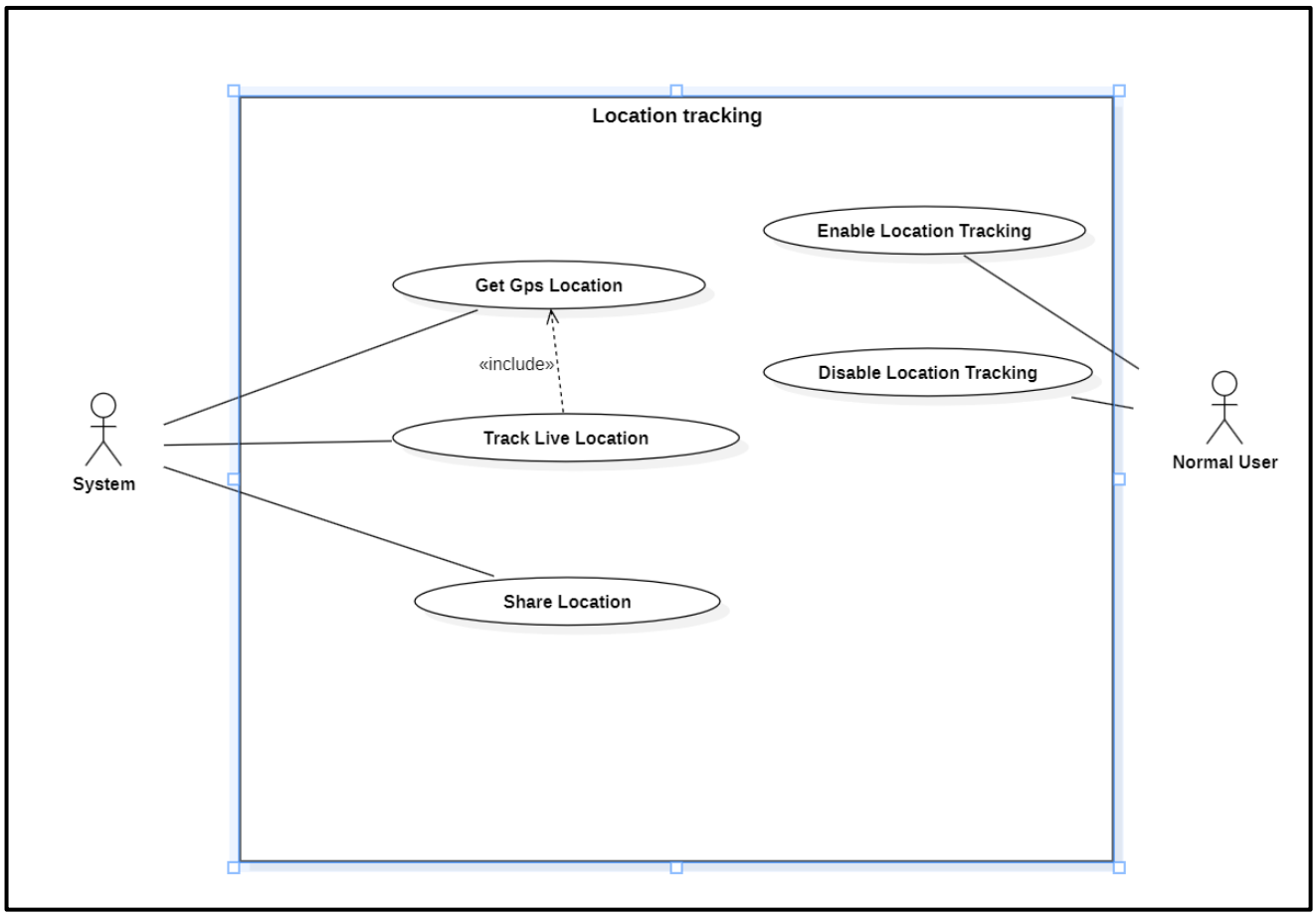
  - Enable Location Tracking
  - Disable Location Tracking
  - Get GPS Location
  - Track Live Location
  - Share Location

- **Relationships :-**

  - Track Live Location includes Get GPS Location to continuously obtain real-time coordinates.
  - Enable Location Tracking allows the system to start location monitoring.
  - Disable Location Tracking stops the location monitoring process when required.

- **Use Case Diagram Description:-**

  - The use case diagram represents how the system tracks and manages the user's location using GPS services.
  - It shows the interaction between the user and the system for enabling, disabling, and sharing location information.


Location tracking
Enable Location Tracking
Get Gps Location
«include»
Disable Location Tracking
Track Live Location
System
Normal User
Share Location

Module 5:- Manual SOS

- **Description :-**
  The Manual SOS module allows the user to manually request emergency assistance during critical situations. This feature is useful when the user feels unsafe or faces an emergency without an accident being detected automatically. Once the SOS is triggered, the system sends an emergency alert along with the current location to emergency contacts, ensuring quick help.

- **Actors :-**

  - **Primary Actor:** Normal User
  - **Secondary Actor**: System

- **Use Cases :-**

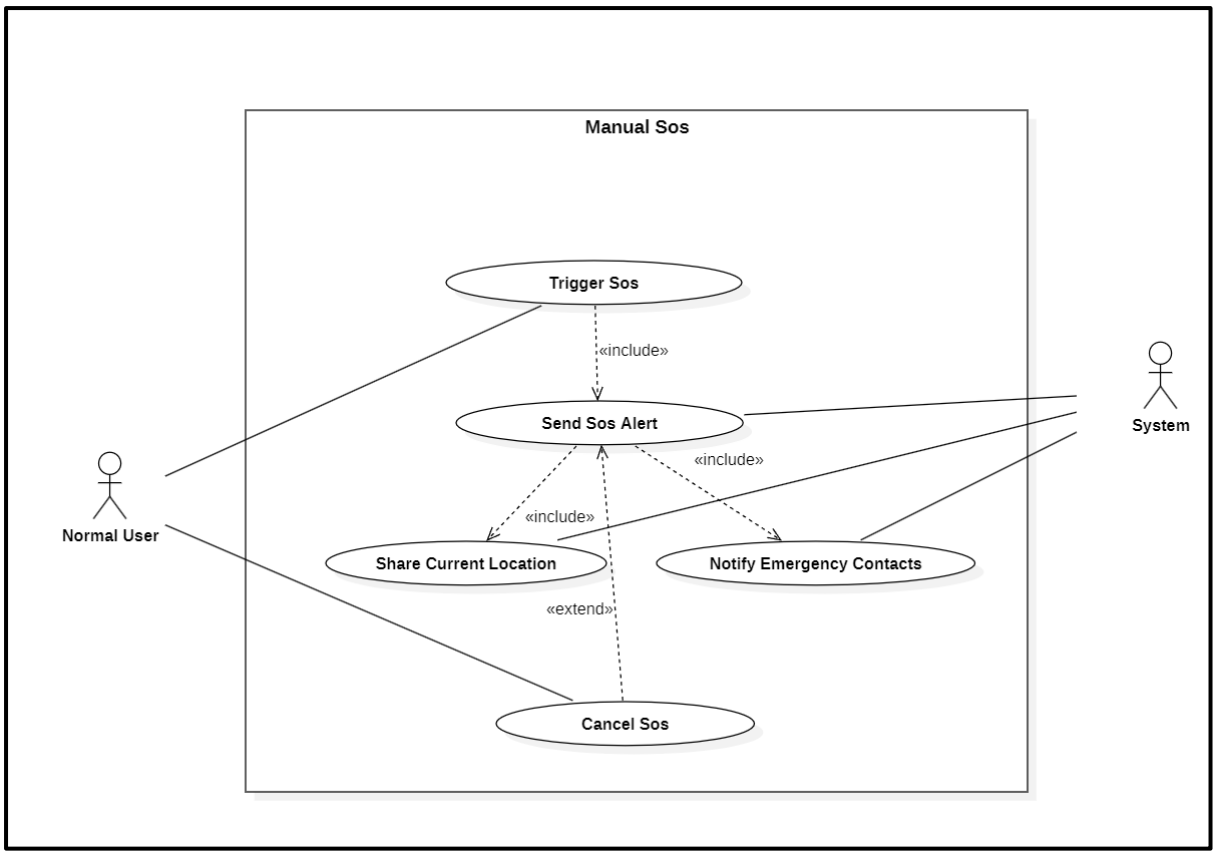
  - Trigger SOS
  - Send SOS Alert
  - Share Current Location
  - Notify Emergency Contacts
  - Cancel SOS

- **Relationships :-**

  - Trigger SOS includes Send SOS Alert to initiate emergency communication.
  - Send SOS Alert includes Share Current Location to provide accurate location details.
  - Send SOS Alert includes Notify Emergency Contacts to inform trusted contacts immediately.
  - Cancel SOS is an optional action that allows the user to stop the SOS alert if triggered accidentally.

➢ Use Case Diagram Description

- The use case diagram illustrates how the user manually triggers an SOS and how the system sends alerts with location details.
- It shows the interaction between the user and the system for handling emergency situations effectively.



Module 6 :- Road Safety & Route Alerts

- **Description** :-

  The Road Safety & Route Alerts module helps users travel safely by identifying unsafe road conditions such as road construction and blocked routes. The system analyzes map and road data to detect potential risks and informs the user in advance. It also suggests safer alternative routes to avoid accidents and delays during travel.

- **Actors :-**

  - **Primary Actor:** System
  - **Secondary Actor:** Normal User

- **Use Cases :-**

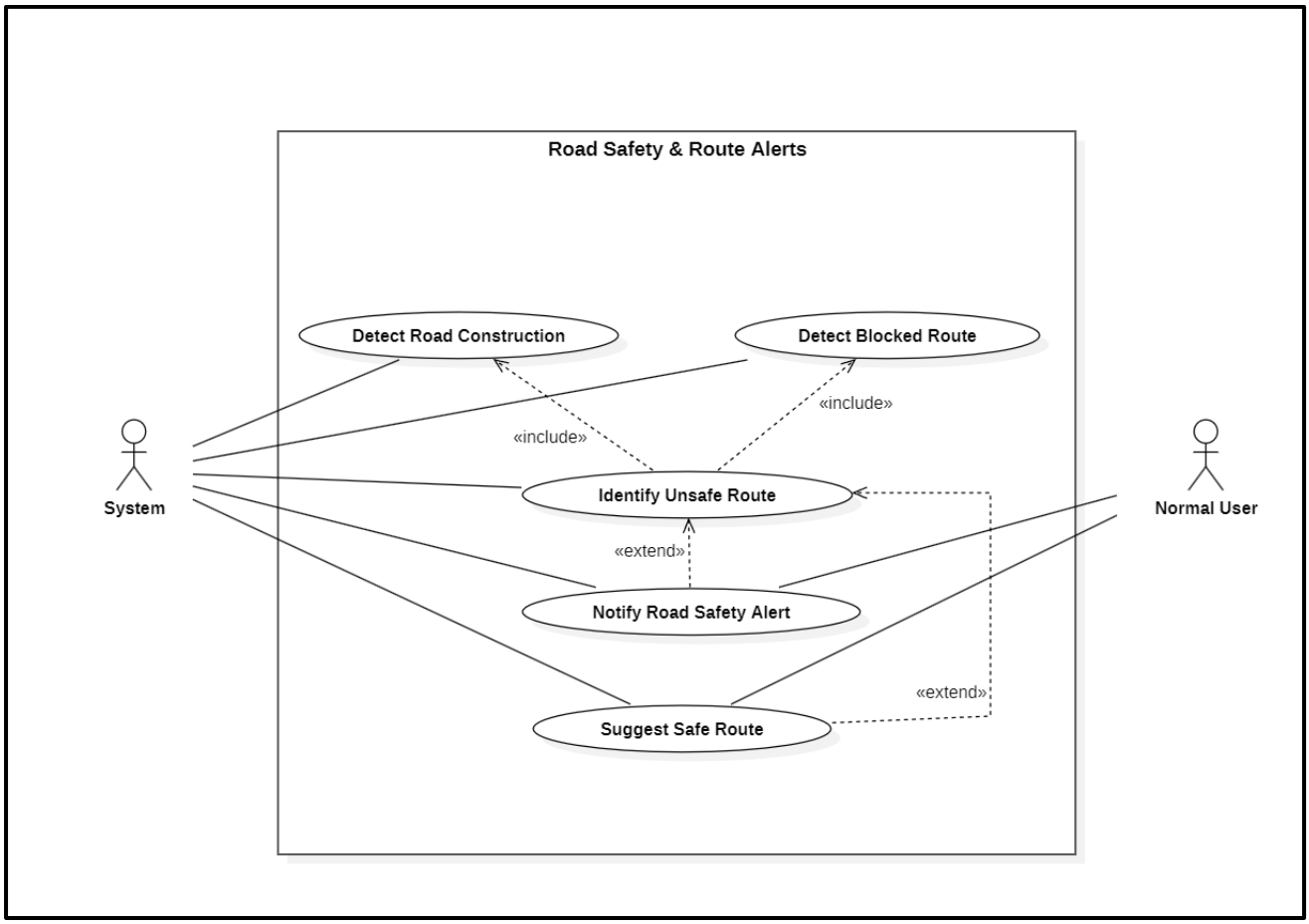
  - Detect Road Construction
  - Detect Blocked Route
  - Identify Unsafe Route
  - Notify Road Safety Alert
  - Suggest Safe Route

- **Relationships :-**

  - Identify Unsafe Route includes Detect Road Construction to check construction-related risks.
  - Identify Unsafe Route includes Detect Blocked Route to analyze route blockages.
  - Notify Road Safety Alert is performed to inform the user about unsafe routes.
  - Suggest Safe Route provides an alternative path when unsafe conditions are detected.
  - Use Case Diagram Description – Module 6 (Road Safety & Route Alerts)
  - The use case diagram represents how the system detects unsafe road conditions and alerts the user during travel.
  - It shows how road construction and blocked routes are analyzed to notify users and suggest safer alternative routes.

➢ Use Case Diagram Description

- The use case diagram represents how the system detects unsafe road conditions and alerts the user during travel.
- It shows how road construction and blocked routes are analyzed to notify users and suggest safer alternative routes.



Module 7 :- Admin & System Management

- **Description** :-
  The Admin & System Management module allows the administrator to manage and monitor the overall system. This module ensures proper administration of user accounts, hospitals or emergency services, and system alert records. It helps maintain system reliability, data accuracy, and smooth operation of the application.

- **Actors :-**

  - **Primary Actor**: Administrator

- **Use Cases :-**

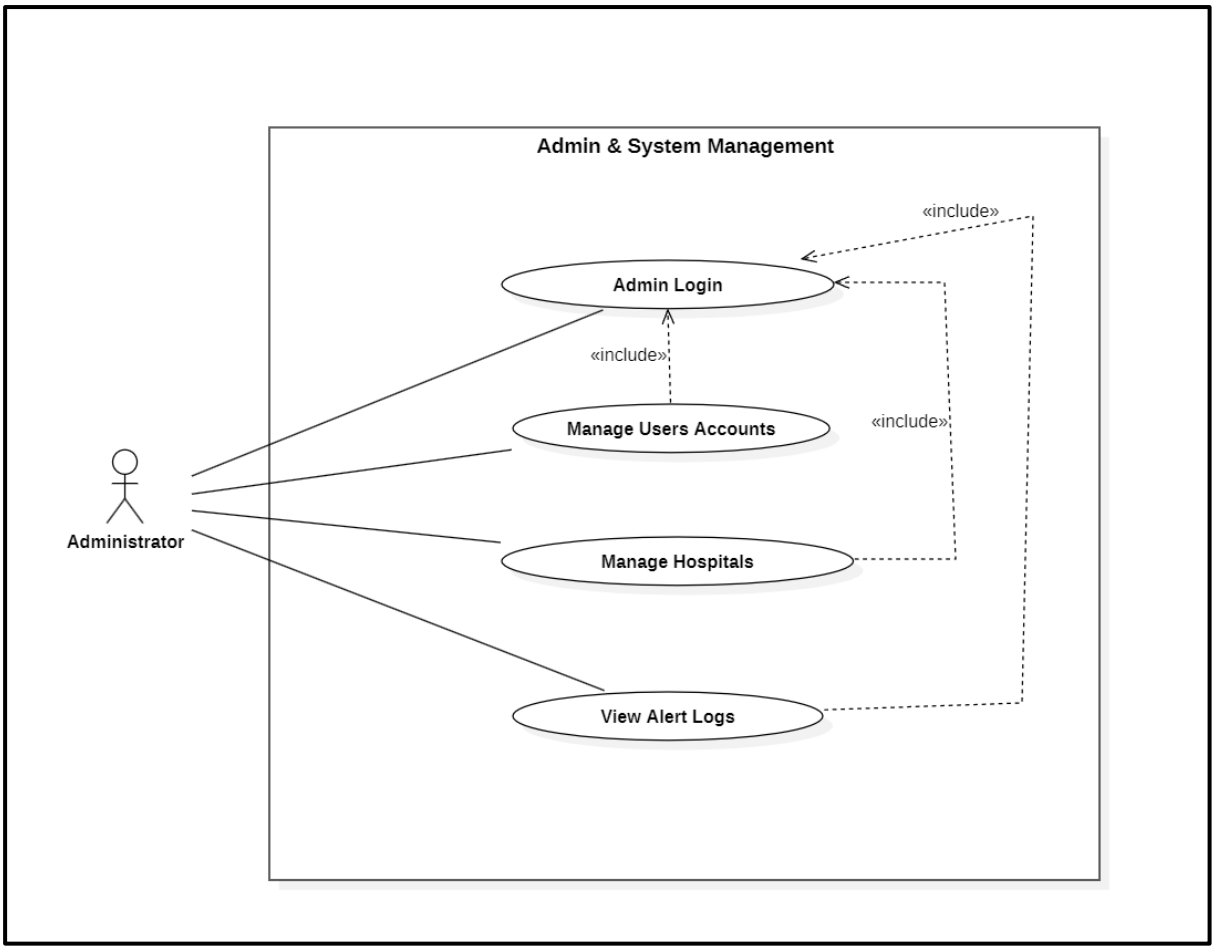
  - Admin Login
  - Manage User Accounts
  - Manage Hospitals / Emergency Services
  - View Alert Logs

- **Relationships :-**

  - Manage User Accounts includes Admin Login to ensure secure access.
  - Manage Hospitals / Emergency Services includes Admin Login to allow authorized management.
  - View Alert Logs includes Admin Login to maintain data security and system integrity.

- **Use Case Diagram Description**

  - The use case diagram shows how the administrator manages users, hospitals, and system alert records after logging into the system.
  - It represents the administrative control and monitoring functions required for proper system maintenance.


Admin & System Management
«include»
Admin Login
«include»
Manage Users Accounts
«include»
Administrator
Manage Hospitals
View Alert Logs

B. CLASS DIAGRAMS

Module 1 :- User Management

- **Description :-**

  This module handles user registration, login, and logout functionalities.It authenticates users and manages access permissions to ensure secure and controlled usage of the application.

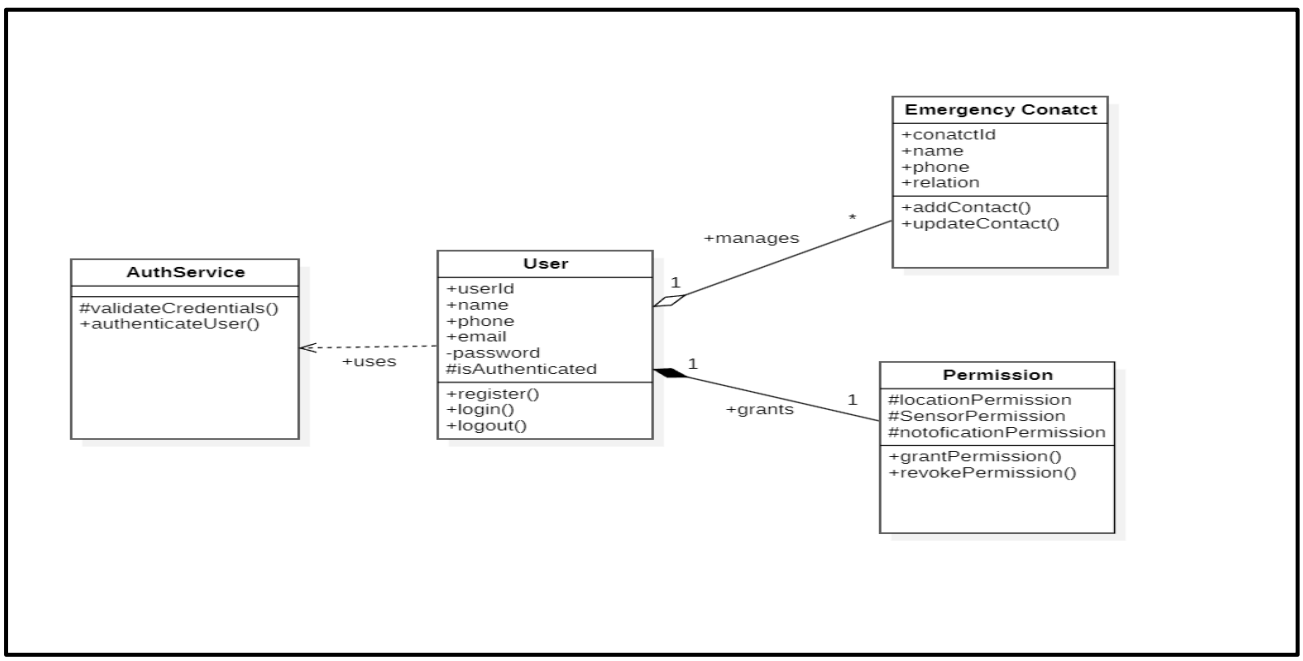
- **Actors :-**

  - **Primary Actor:** User

- **Classes :-**

  - User
  - Registration
  - Authentication
  - Permission

Class Diagram:-



Module 2 :- User Management

- **Description :-**

  This module continuously monitors mobile sensor data such as accelerometer and gyroscope values to detect road accidents automatically.It triggers an alert mechanism when abnormal sensor readings indicate a possible accident.

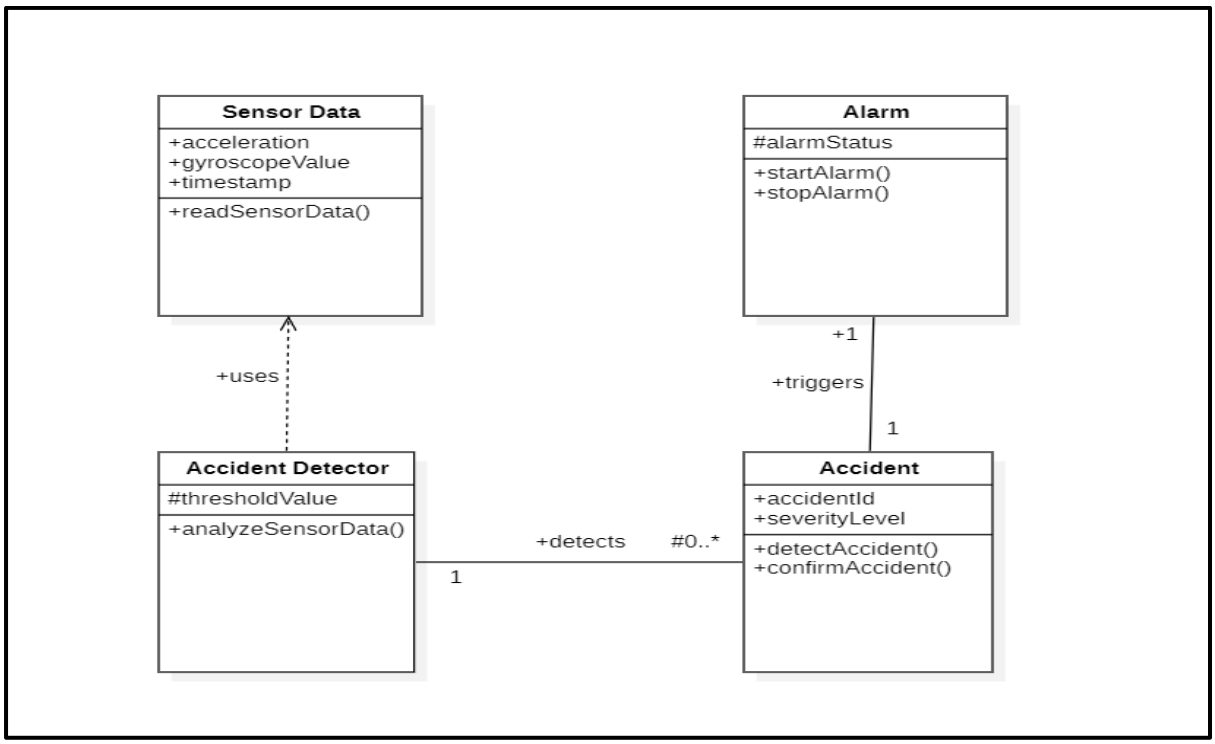
- **Actors :-**

  - **Primary Actor:** System

- **Classes :-**

  - Sensor Data
  - Accident Detector
  - Accident
  - Alarm

Class Diagram:-



Module 3 :- Emergency Alert & Notification

- **Description :-**

  This module sends emergency alerts to registered emergency contacts and nearby hospitals when an accident or SOS is detected.It ensures timely communication by sharing alert messages along with location details.

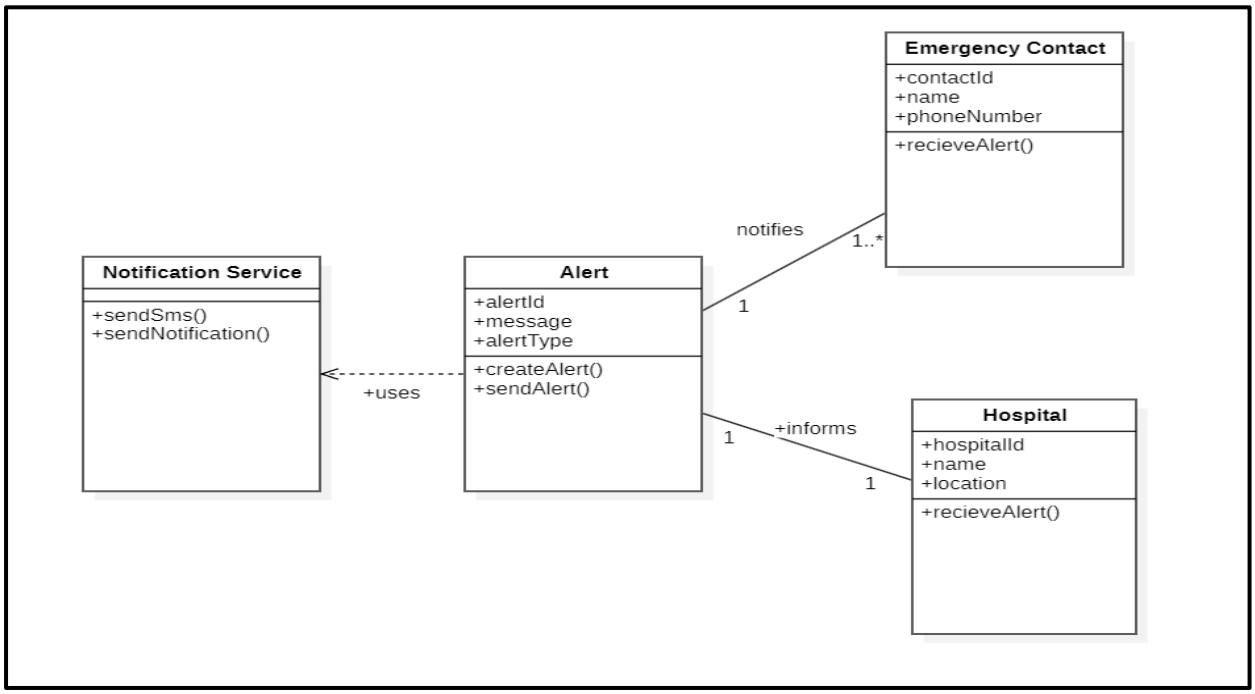
- **Actors :-**

  - **Primary Actor:** System
  - **Secondary Actor**: Emergency Contact,Hospital

- **Classes :-**

  - Alert
  - Emergency Contact
  - Hospital
  - Notification Service

Class Diagram:-



Module 4 :- Location Tracking

- **Description :-**

  This module tracks the real-time location of the user using GPS services.It provides accurate location information to other modules such as accident detection, SOS, and emergency alerts.

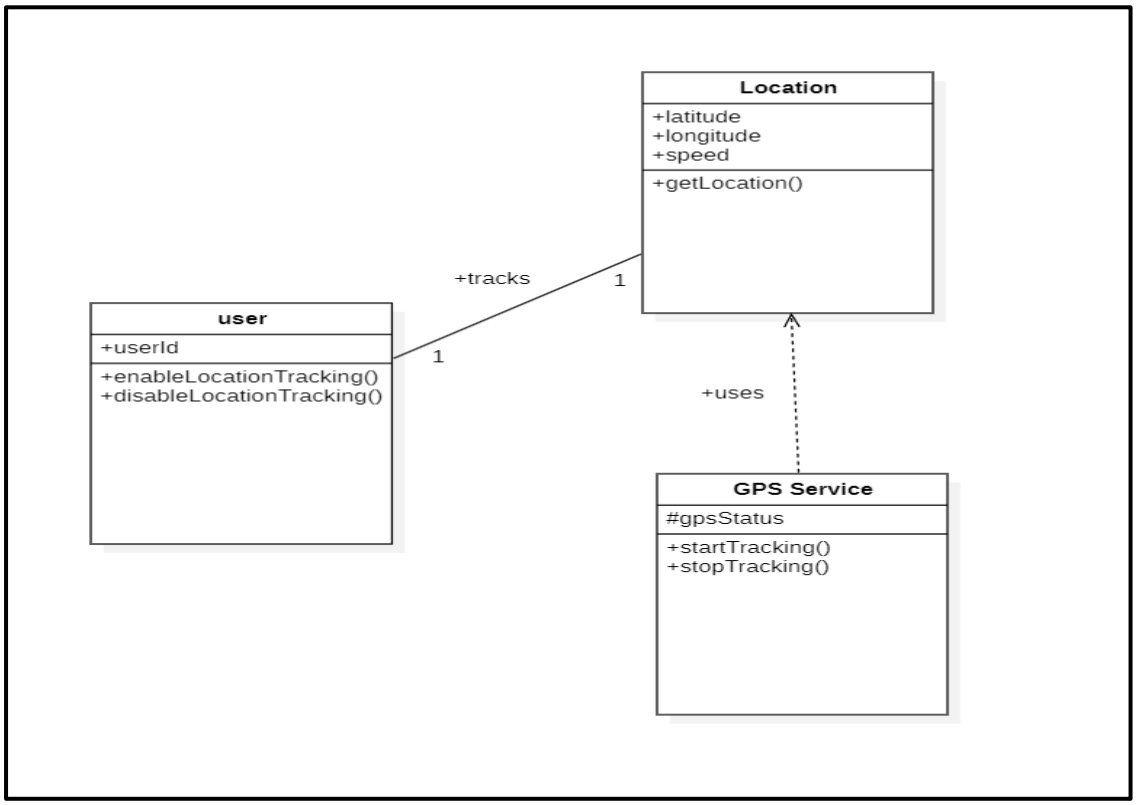
- **Actors :-**

  - **Primary Actor:** User, System

- **Classes :-**

  - Location
  - GPSService
  - User

Class Diagram:-



Module 5:- Manual SOS

- **Description :-**

  This module allows the user to manually trigger an SOS alert during emergency situations.It sends the user's current location along with an alert message to emergency contacts.

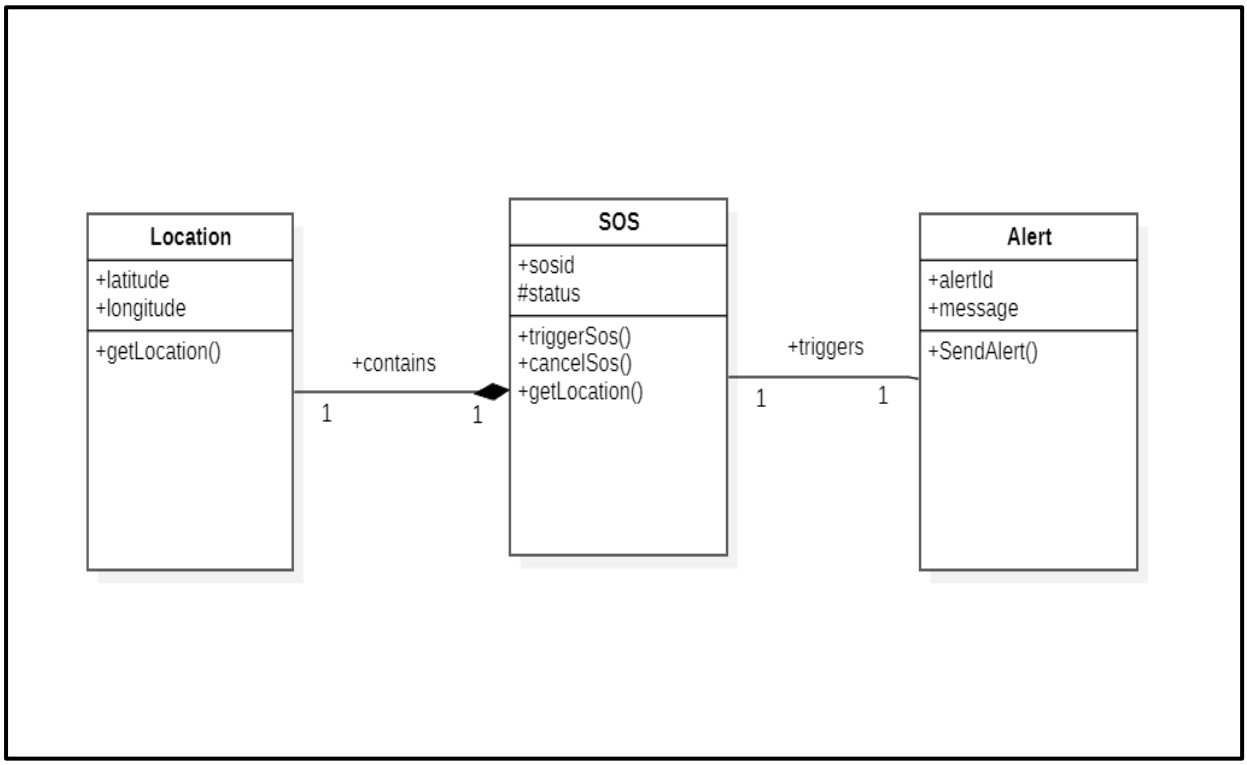
- **Actors :-**

  - **Primary Actor:** user
  - **Secondary Actor**: System

- **Classes :-**

  - SOS
  - Alert
  - Location

Class Diagram:-



Module 6:- Road Safety & Route Alerts

- **Description :-**

  This module detects road construction, blocked routes, and unsafe paths using map data.It notifies users with safety alerts and suggests alternative safe routes for travel.

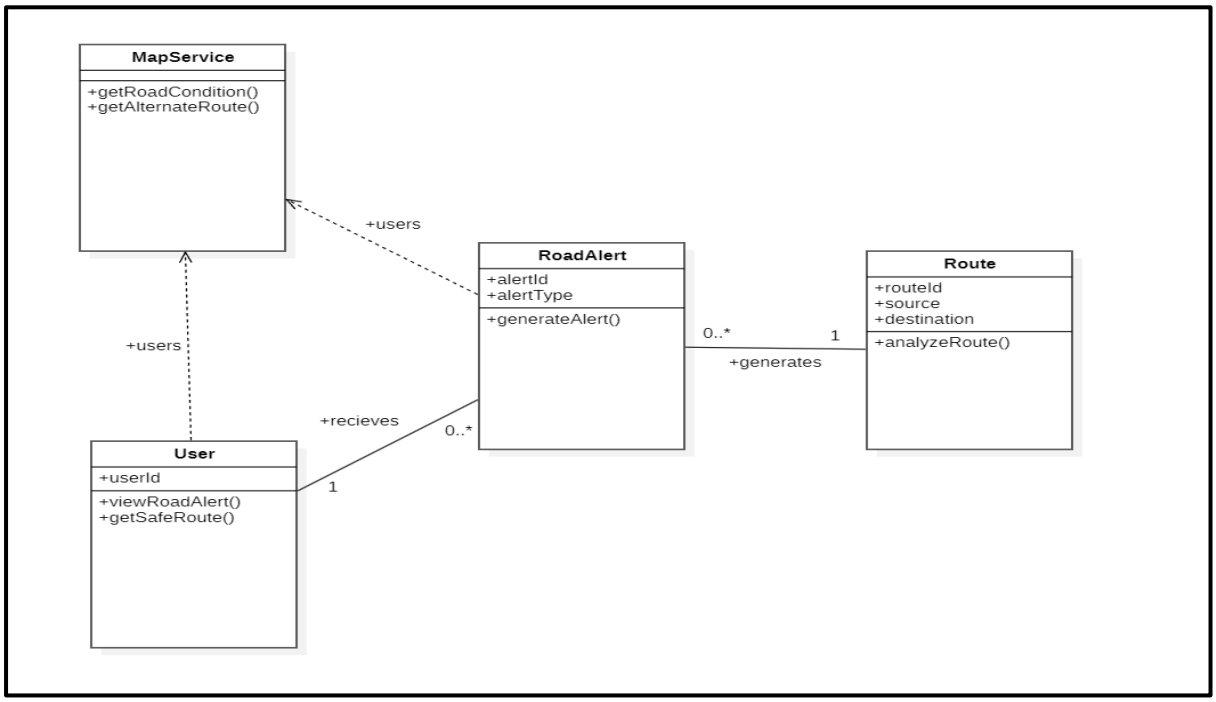
- **Actors :-**

  - **Primary Actor:** System,User

- **Classes :-**

  - Route
  - Road Alert
  - Map Service
  - User

Class Diagram:-



Module 7:- Admin & System Management

- **Description :-**

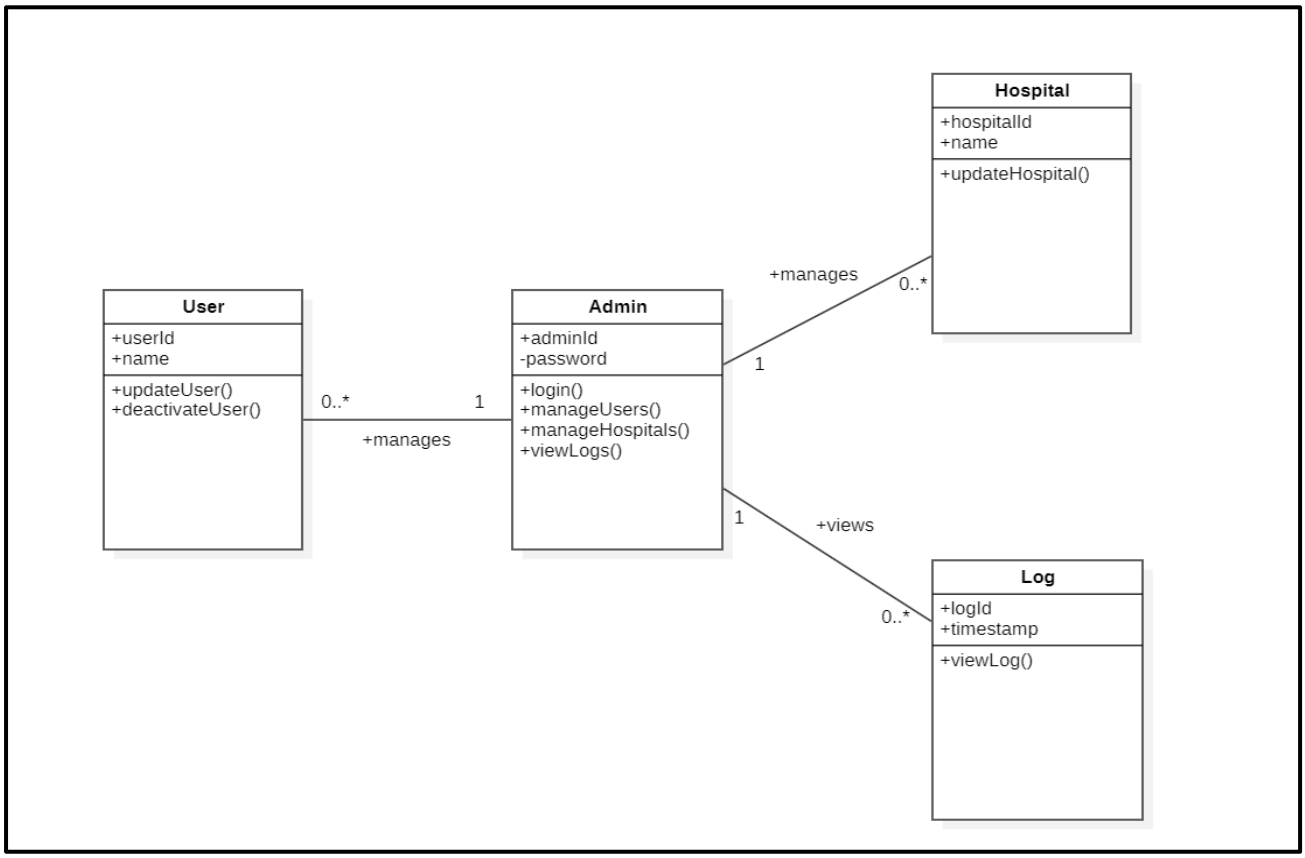
  This module allows the administrator to manage users, hospitals, and system data.It also enables monitoring of alert logs and overall system maintenance

- **Actors :-**

  - **Primary Actor:** Administrator

- **Classes :-**

  - Admin
  - User
  - Hospital
  - Log



C. STATE CHART DIAGRAM:-

i) Module Name: Manual SOS

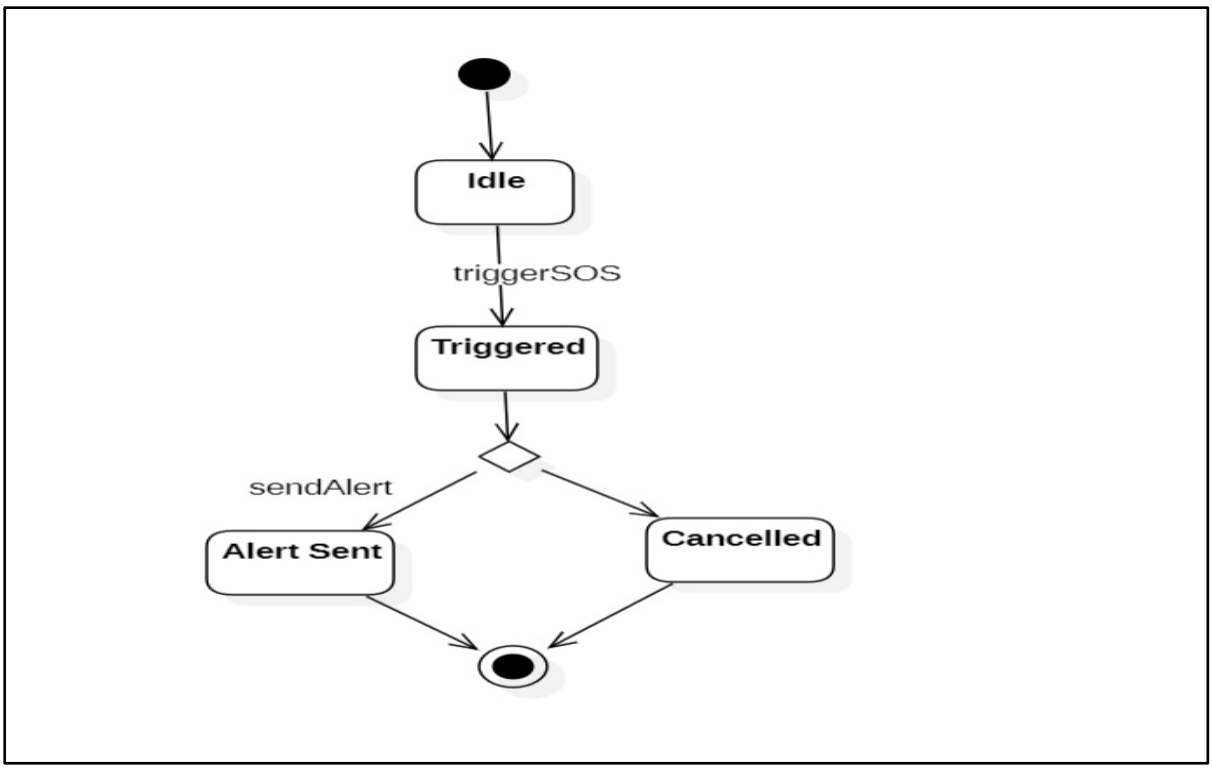
1. SOS Object

- **Description:**

  This state diagram shows the lifecycle of an SOS request in the emergency alert system.It represents how the SOS object changes state when triggered, processed, or cancelled by the user.

States Involved:

1. Idle
2. Triggered
3. Alert Sent
4. Cancelled



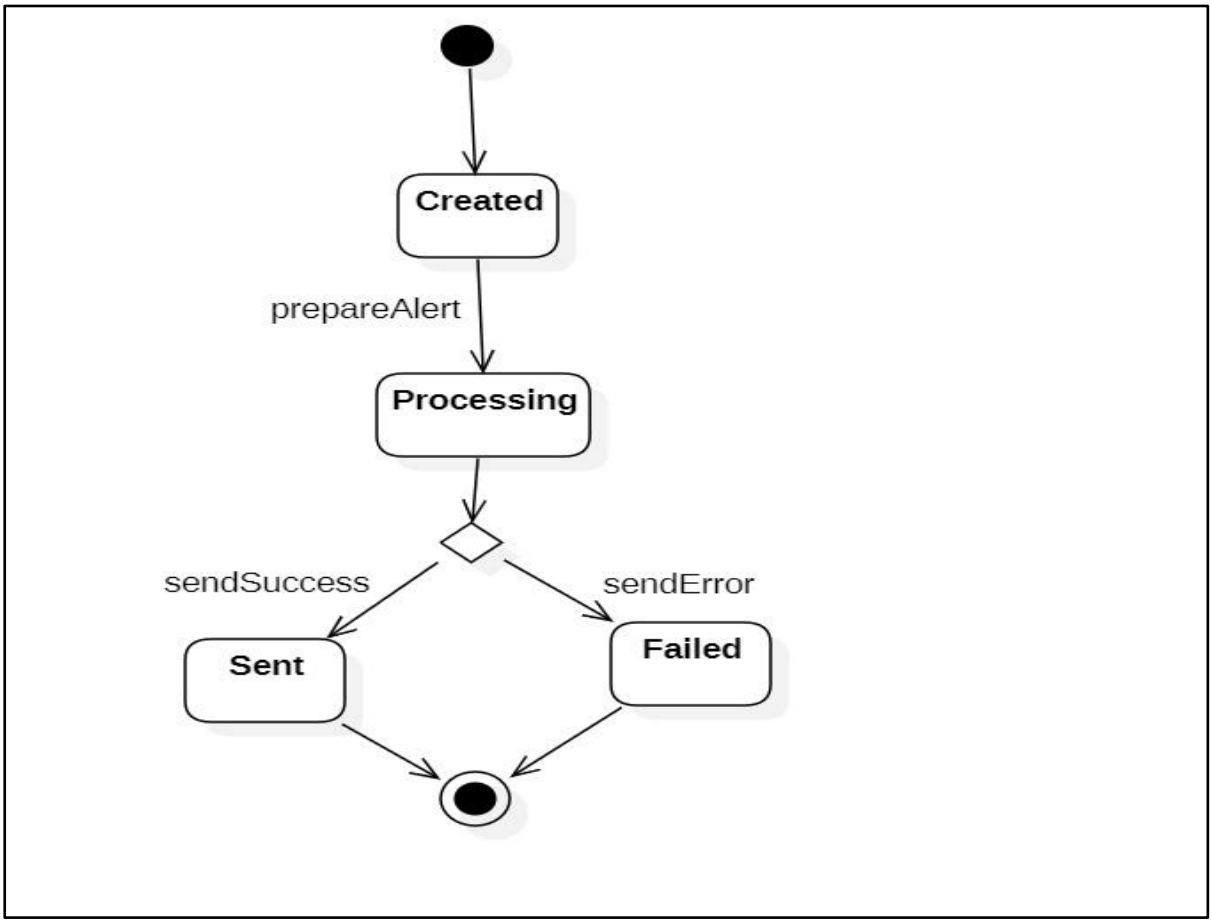
2.Alert Object

- **Description:**

  This diagram shows the internal state changes of an emergency alert generated after SOS activation.It models how an alert is created, processed, delivered, or fails.

States Involved:

1.Created
2.Processing
3.Sent
4.Failed



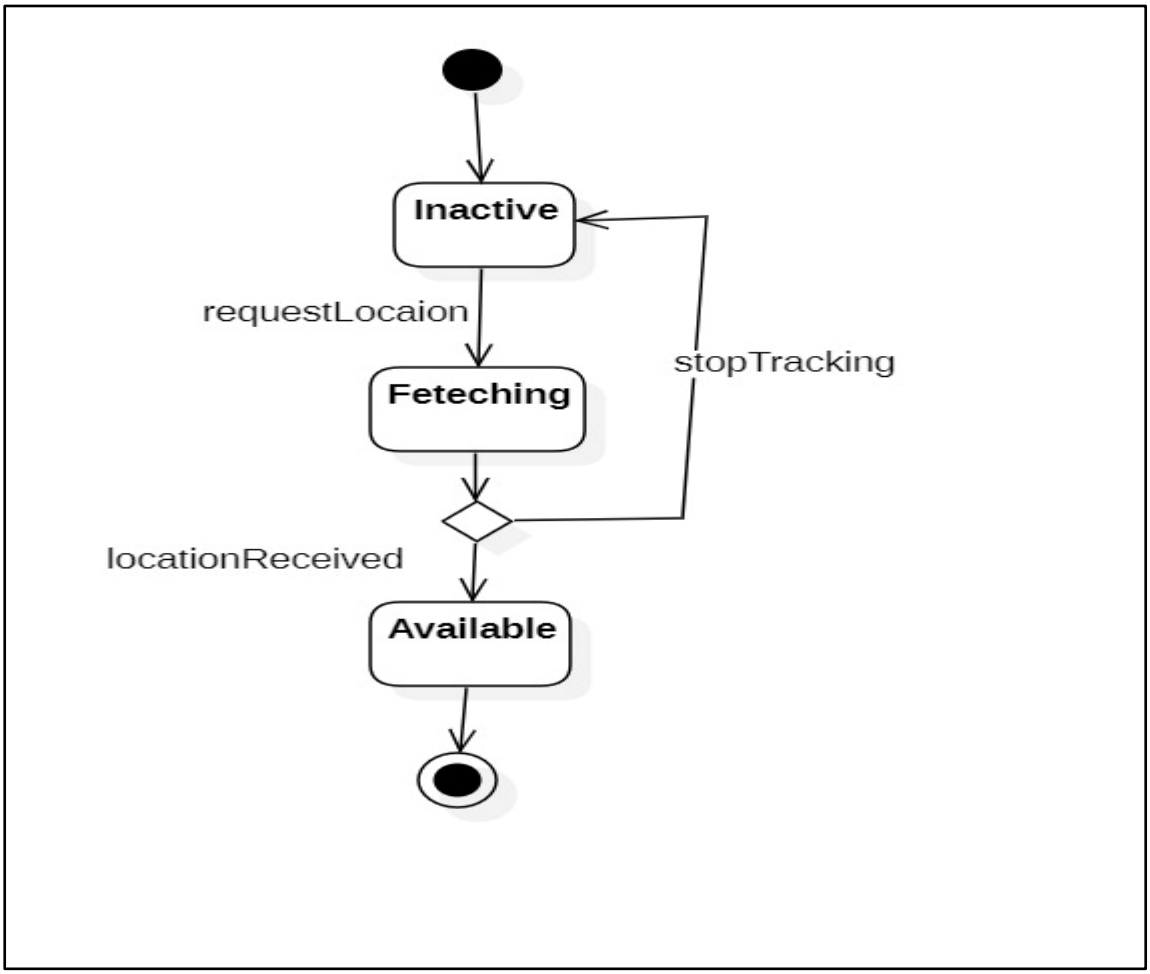
3. Location Object

- **Description:**

  This diagram shows the states of the location service used during emergency tracking.It represents how GPS data is requested, fetched, and made available to the system.

States Involved:

1.Inactive
2.Fetching
3.Available

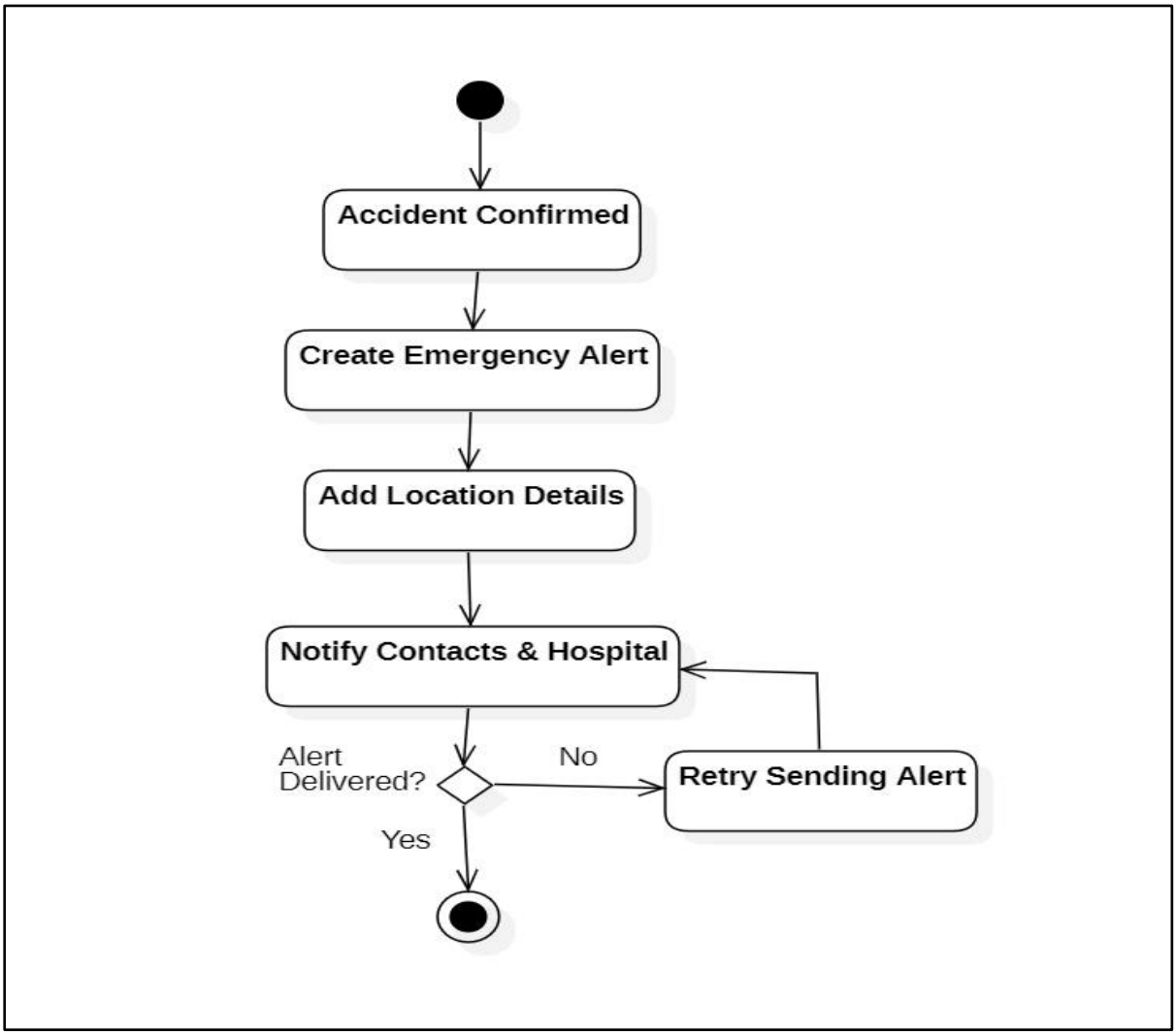


D.ACTIVITY DIAGRAMS:

Module Name: Accident Detection

- **Description:**

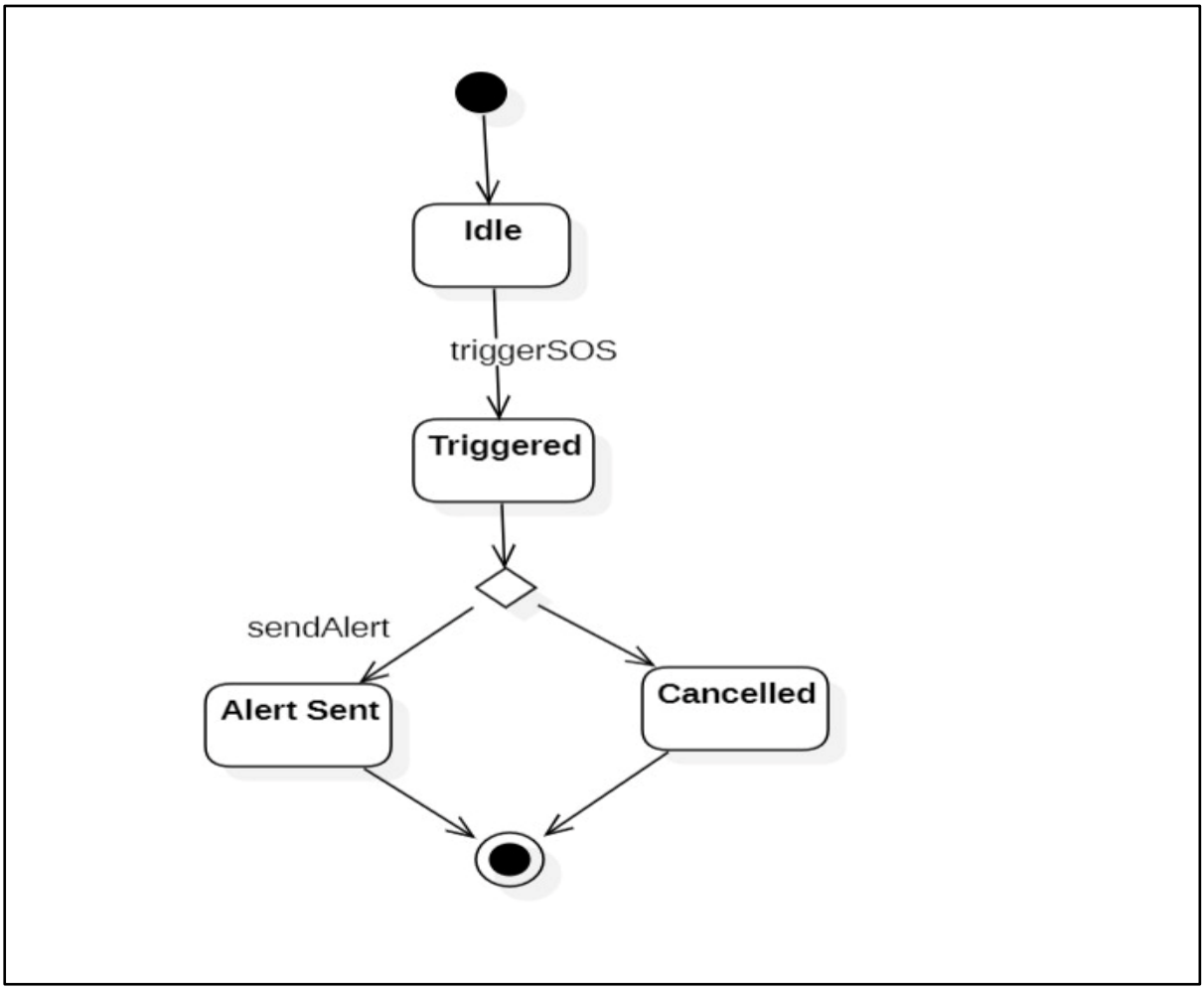
  The Accident Detection module continuously monitors sensor data from the mobile device to identify abnormal movement or impact. It analyzes sensor values and automatically triggers an alarm when a possible accident is detected. This module ensures fast detection and immediate response to emergencies.



ii) Module Name: Emergency Alert & Notification

- **Description:**

  The Emergency Alert and Notification module sends automatic alerts to emergency contacts and nearby hospitals after an accident is confirmed. It shares the user's live location and emergency message to ensure quick assistance. This module helps reduce response time and improves user safety.



E.Sequence Diagram:-

i)Module 1:- User Management

- **Description:-**

  The User management sequence diagram depicts the process of Depicts the process of user registration, permission validation, and Emergency contact addition. It also shows secure logout by invalidating the user session.

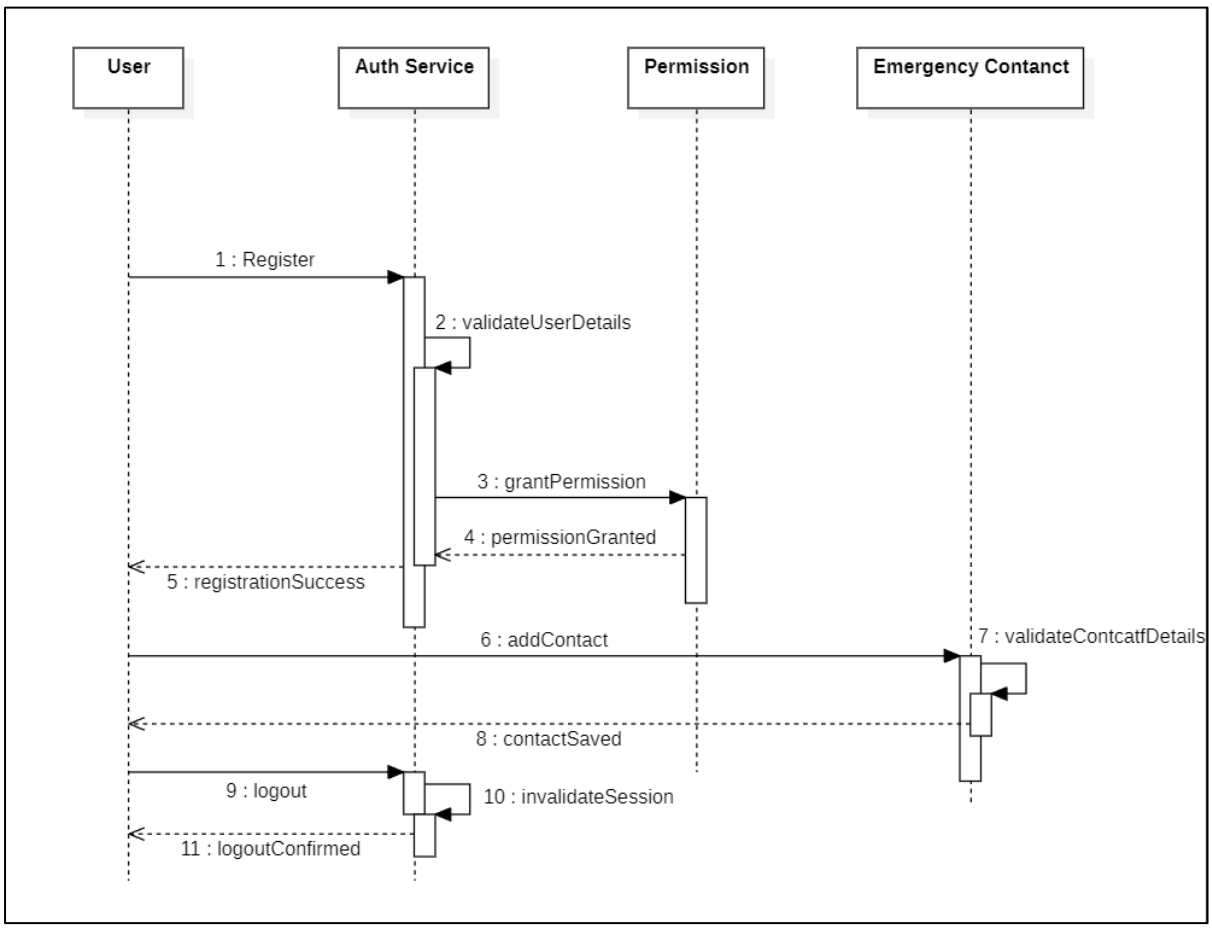


Fig: User Management

ii) Module 2:- Emergency Alert Management

- **Description** :-

  The Emergency Alert Management sequence diagram shows how an alert is triggered and sent to emergency contacts and hospitals. It ensures that the alert is successfully delivered and confirmed for timely emergency response.

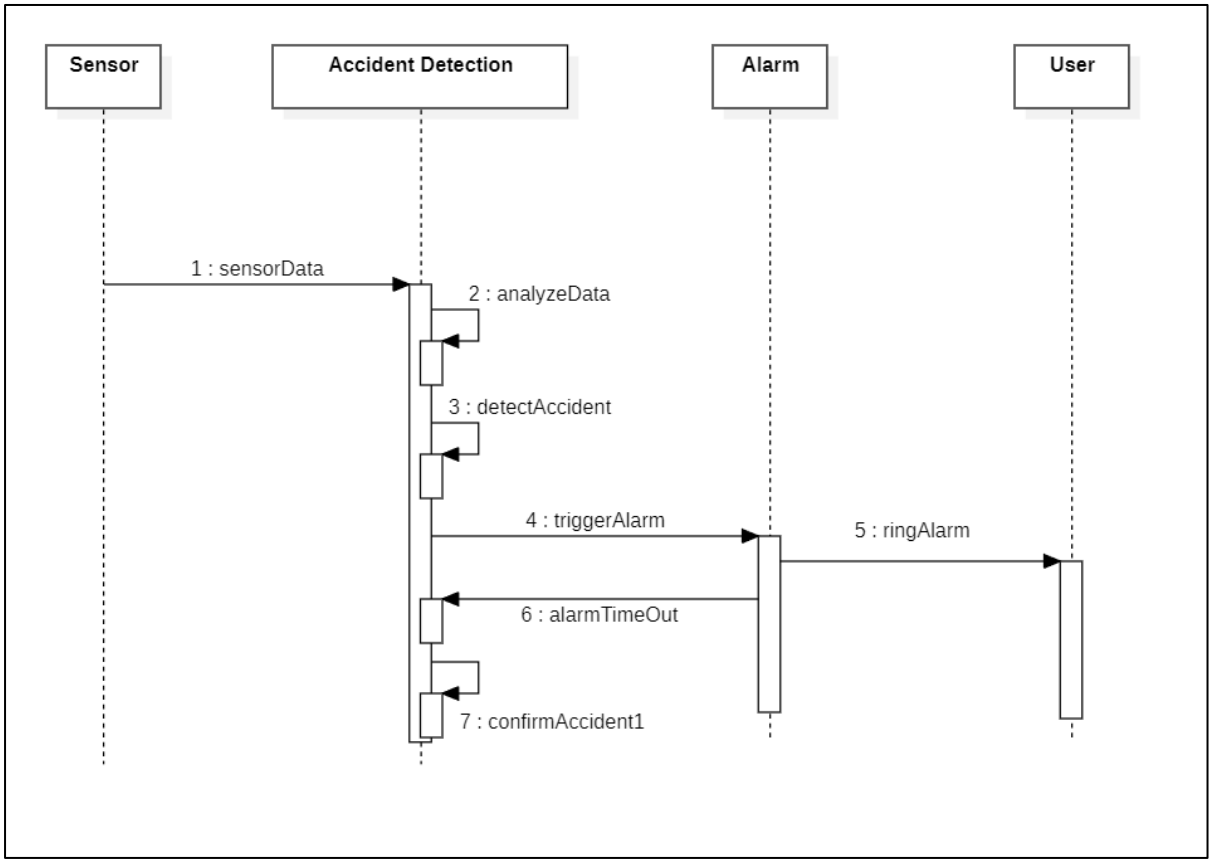


Fig: Emergency Alert Management

Module 3:-Road Safety Management

- **Description:-**

  The Road Safety and Route Alerts sequence diagram illustrates how routes are planned using map services and road alerts are checked. It also shows how users are notified about unsafe roads or route updates in real time.

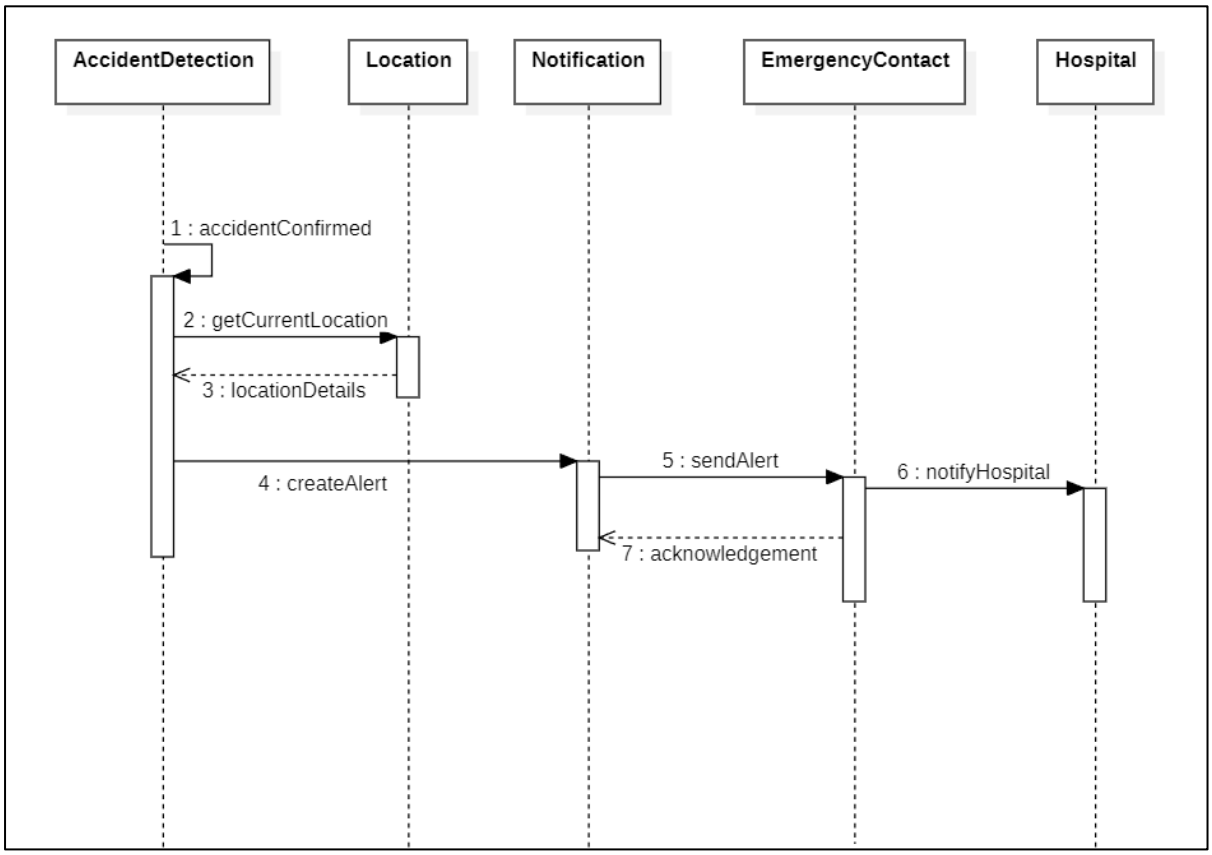


Fig: Road Safety Management

F. COLLABORATION DIAGRAM:-

i)Module 1: User Management

- **Description:-**

  The User management sequence diagram depicts the process of Depicts the process of user registration, permission validation, and

Emergency contact addition. It also shows secure logout by invalidating the user session.

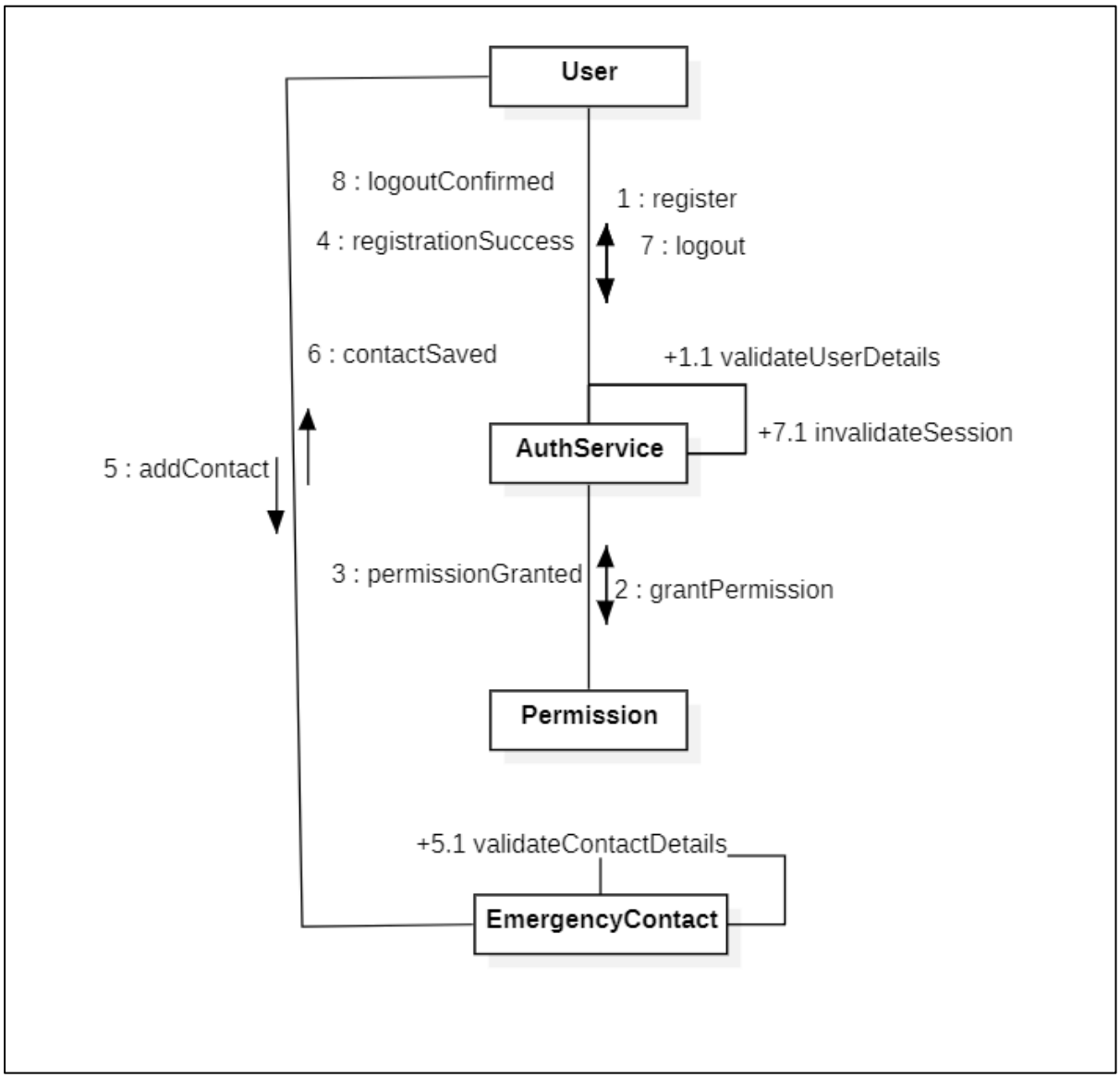


Fig: User Management

ii) Module 2:- Emergency Alert Management

- **Description** :-

  The Emergency Alert Management sequence diagram shows how an alert is triggered and sent to emergency contacts and hospitals. It ensures that the alert is successfully delivered and confirmed for timely emergency response.

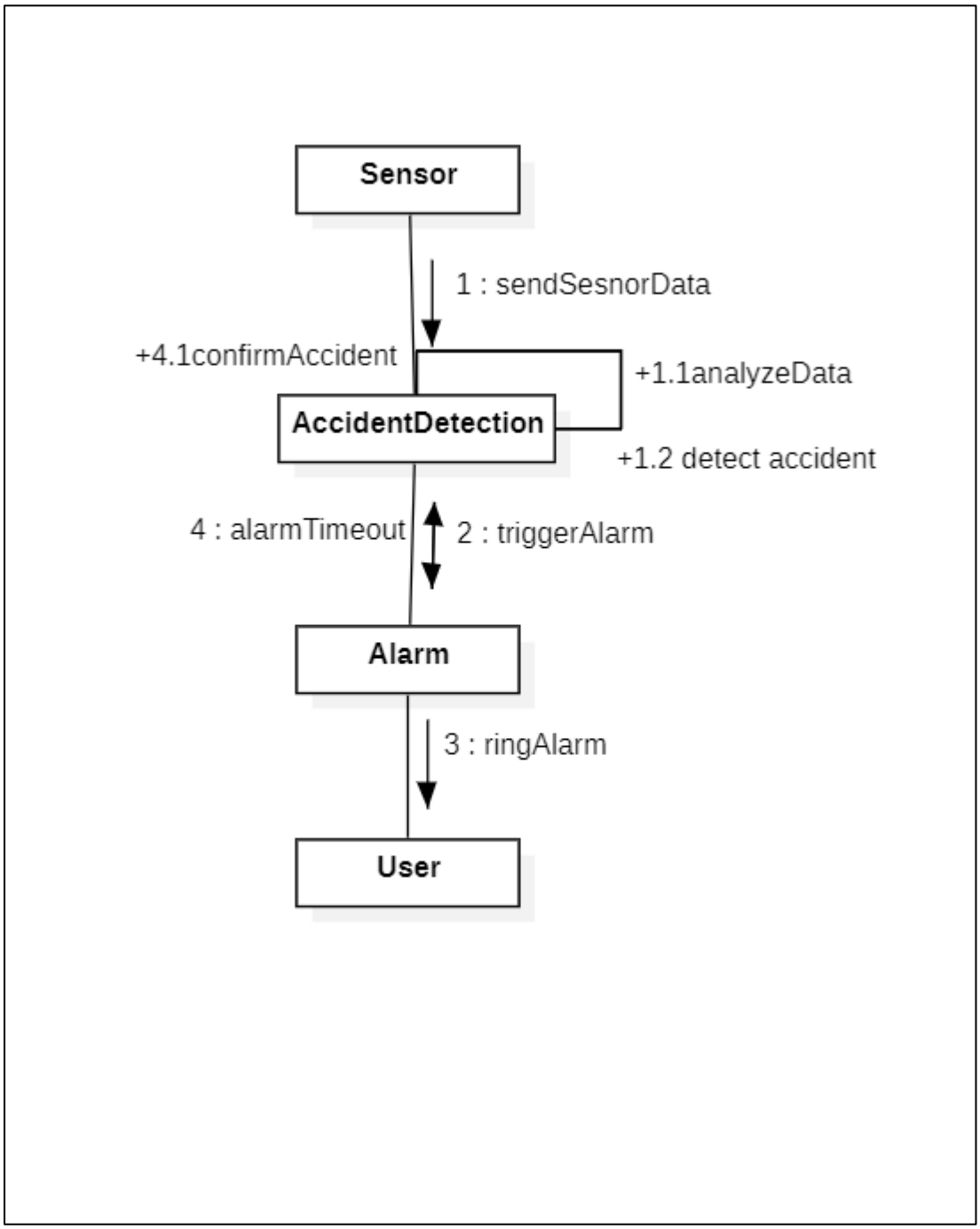


Fig:Emergency Alert Management

Module 3:-Road Safety Management

- **Description:-**

  The Road Safety and Route Alerts sequence diagram illustrates how routes are planned using map services and road alerts are checked. It also

shows how users are notified about unsafe roads or route updates in real time.

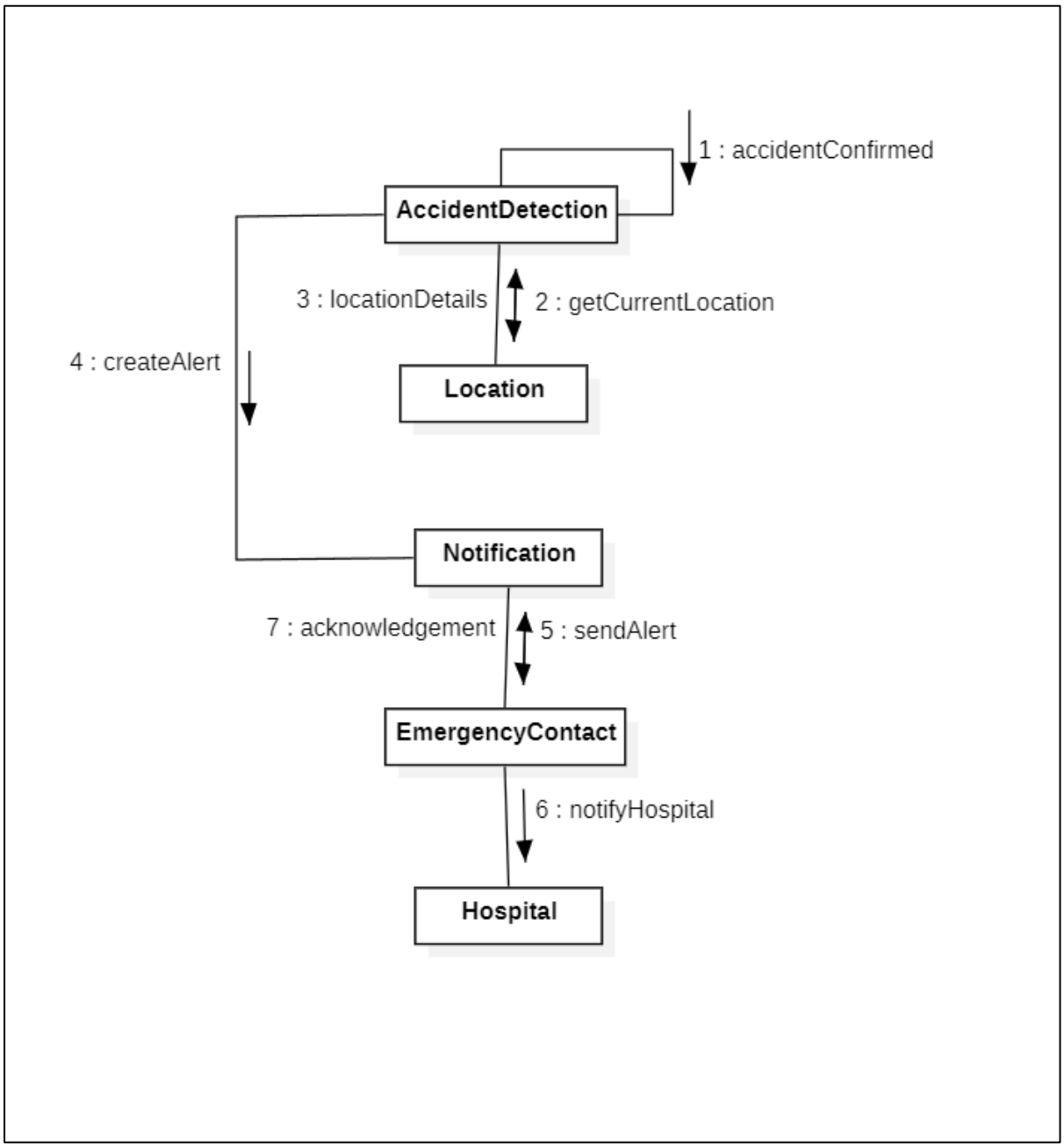


Fig: Road Safety Management

3.2 ER Diagram:-

The ER diagram represents the data structure of the Smart Accident Detection and Emergency Alert System. It shows the main entities such as User, Accident, Alert, Location, Emergency Contact, and Hospital along with their attributes and relationships. The diagram explains how users manage emergency

contacts, how accidents occur at locations, generate alerts, and how alerts notify hospitals for emergency response.

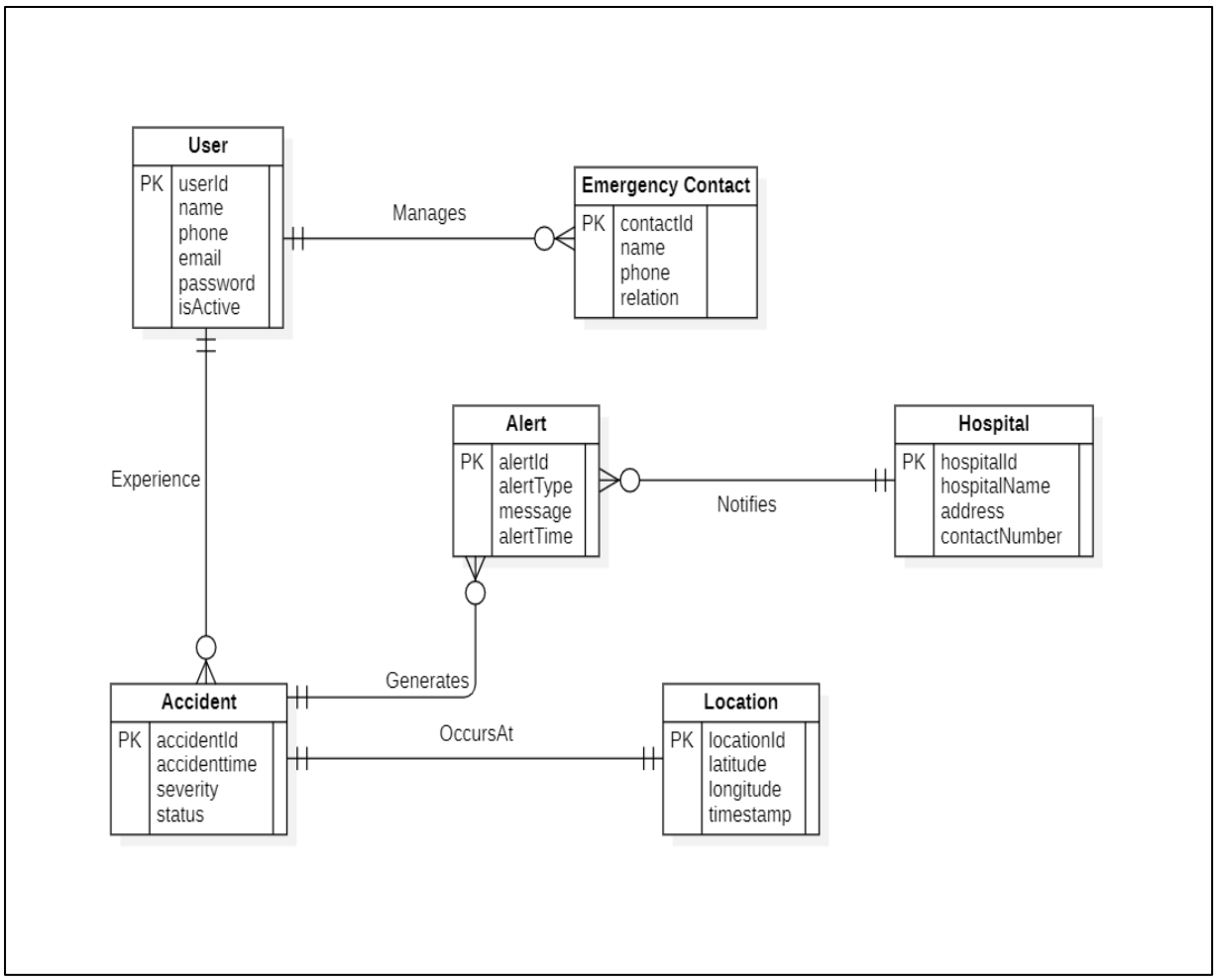


Fig:ER-Diagram

3.3 DFD Diagram:

The Data Flow Diagram (DFD) illustrates how data moves within the Smart Accident Detection and Emergency Alert System. It shows the interaction between users, system processes, data stores, emergency contacts, and hospitals,

highlighting how accident data is processed, location information is retrieved, and emergency alerts are generated and communicated.

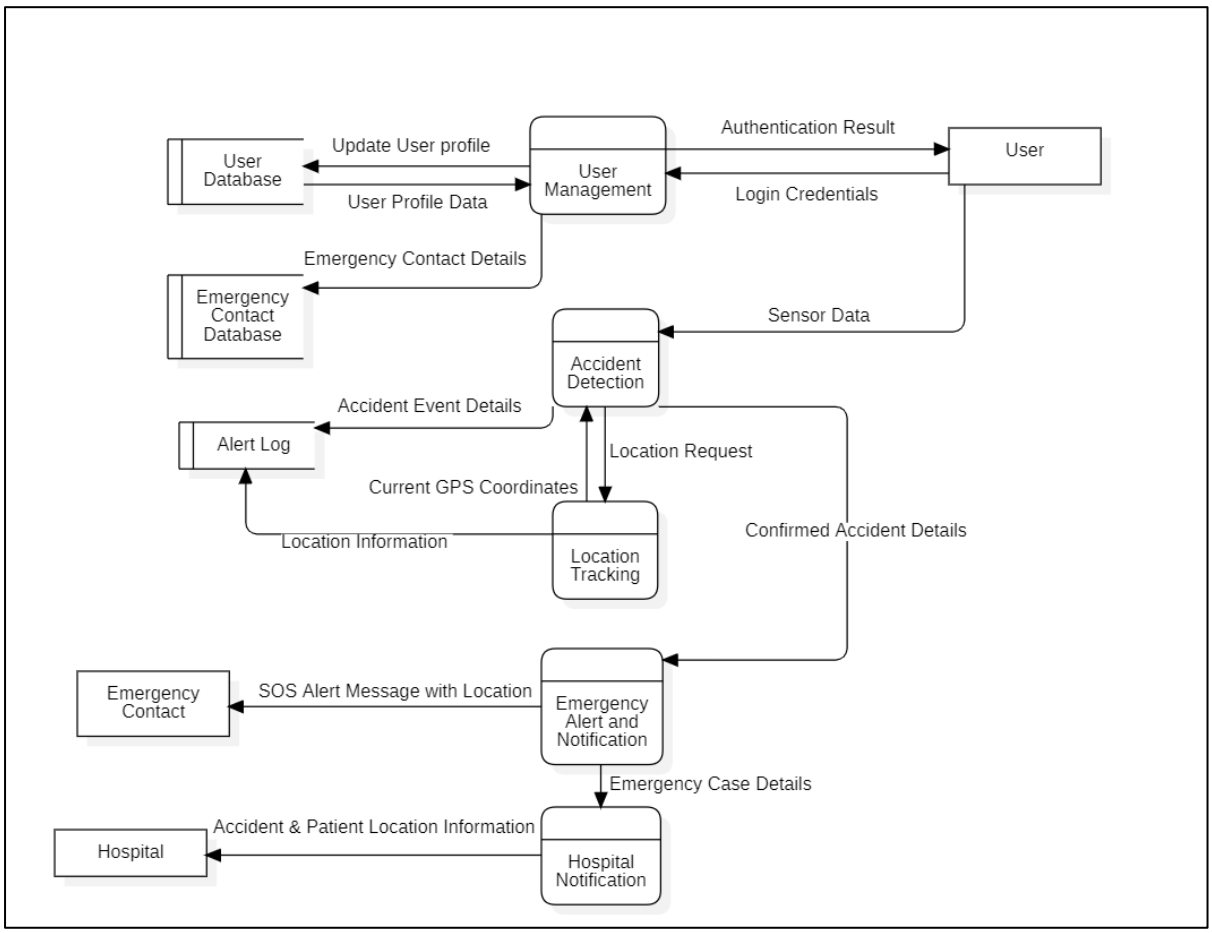


Fig:Data Flow Diagram

4.EXTERNAL INTERFACE REQUIREMENTS

This section explains how users and external systems interact with the Smart Accident Detection and Emergency Alert System.

➢ 4.1 User Interfaces:

The system provides a user-friendly Android mobile application that can be easily used by vehicle riders and drivers.

The application allows users to:

- Register and log in securely.
- Add and manage emergency contact details.
- Grant required permissions such as location, sensor, and notification access.
- Automatically detect accidents using mobile sensors.
- Send emergency alerts along with live location details.
- Use a manual SOS button during emergency situations.
- Receive alerts about unsafe roads, construction zones, and blocked routes.
- View alert status and confirmation messages.
- Allow administrators to manage users, hospitals, and system logs through an admin interface.

➢ 4.2 Hardware Interfaces:

The system runs on Android smartphones with basic hardware support.
It requires:

- GPS for location tracking
- Accelerometer and gyroscope sensors for accident detection
- Internet connectivity for maps and notifications
- SMS support for emergency message delivery

➢ 4.3 Software Interfaces:

The system interacts with different software components to work smoothly.
These include:

- Android operating system

- Map services such as Google Maps for tracking location and routes
- Database systems to store user details, emergency contacts, accident records, and alert logs
- Notification and SMS services for sending emergency alerts
- Backend services for processing accident detection and alert generation

➢ 4.4 Communication Interfaces:

- The system uses secure communication methods to share data safely.
- Location and alert information are sent over secure internet connections.
- Emergency alerts are delivered using notifications and SMS services.
- Communication with map services and backend servers is protected to ensure user data privacy.
- All sensitive user and location information is handled securely.

5. OTHER NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality, performance, safety, and operational constraints of the Smart Accident Detection and Emergency Alert System. These requirements define how well the system should work, rather than what functions it performs.

➢ 5.1 Performance Requirements:

- The system shall detect accidents and trigger alerts within a few seconds of sensor data analysis.
- Emergency alerts shall be sent to contacts and hospitals without noticeable delay.
- The application shall operate efficiently in the background without affecting normal smartphone usage.
- The system shall support multiple users sending alerts simultaneously.
- Location tracking and map services shall update in near real-time during emergencies.

➢ 5.2 Safety Requirements:

- The system shall minimize false accident detection by using sensor thresholds and confirmation mechanisms.
- A countdown alarm shall be provided to allow users to cancel false accident alerts.
- Emergency alerts shall not be sent unless an accident is confirmed.
- User and accident data shall not be lost during unexpected application crashes.
- The system shall maintain logs of accident events and emergency alerts.
- The application shall ensure that incorrect or incomplete alerts are not sent to hospitals.
- Stored accident and alert records shall remain consistent even during system failures.

➢ 5.3 Security Requirements:

- The system shall provide secure user authentication using login credentials.
- User passwords shall be stored in encrypted format.

- Location and personal data shall be accessible only to authorized users and services.
- Communication between the application, map services, and backend servers shall be secure.
- Unauthorized access to emergency contact and hospital information shall be prevented.
- The system shall request user permission before accessing sensors and location data.

➢ 5.4 Software Quality Attributes:

1. Reliability:

- The system shall function continuously with minimal failure during travel.
- Accident detection and alert services shall work consistently when required.

2. Usability:

- The application shall provide a simple and easy-to-use interface.
- Emergency features such as SOS and alerts shall be accessible with minimal user interaction.

3. Scalability:

- The system shall support an increase in the number of users, emergency contacts, and hospitals in the future.

4. Maintainability:

- The system shall be easy to update and enhance with new safety features.
- Bugs and issues shall be easy to identify and fix.

5. Availability:

- The system shall be available whenever the user is travelling, provided network connectivity is available.
- Emergency alert functionality shall work even in limited network conditions where possible.

➢ 5.5 Business Rules:

- Users must register and log in before using accident detection features.
- Emergency contacts must be added before alerts can be sent.
- Accident alerts shall be generated only after confirmation by the system.
- Users may cancel alerts during the false alarm countdown period.
- Hospitals shall receive alerts only for verified accident cases.
- User location data shall be shared only during emergencies.
- The system administrator shall have full control over system configuration and monitoring.

6. Testing

6.1 White Box Testing:

Sample Code-1

Module-2: Accident Detection

```
def detect_accident(speed, impact):

    if speed > 60 and impact == True:
        return True
    else:
        return False


# Main Program
speed = int(input("Enter vehicle speed: "))
impact = input("Impact detected (True/False): ").lower() ==
"true"

result = detect_accident(speed, impact)

if result:
    print("Accident Detected")
else:
    print("No Accident")
```

6.1.1 Flow Graph Notation

Control Flow Graph represents the logical flow of the program.

Nodes represent statements or decisions

Edges represent the flow of control between nodes

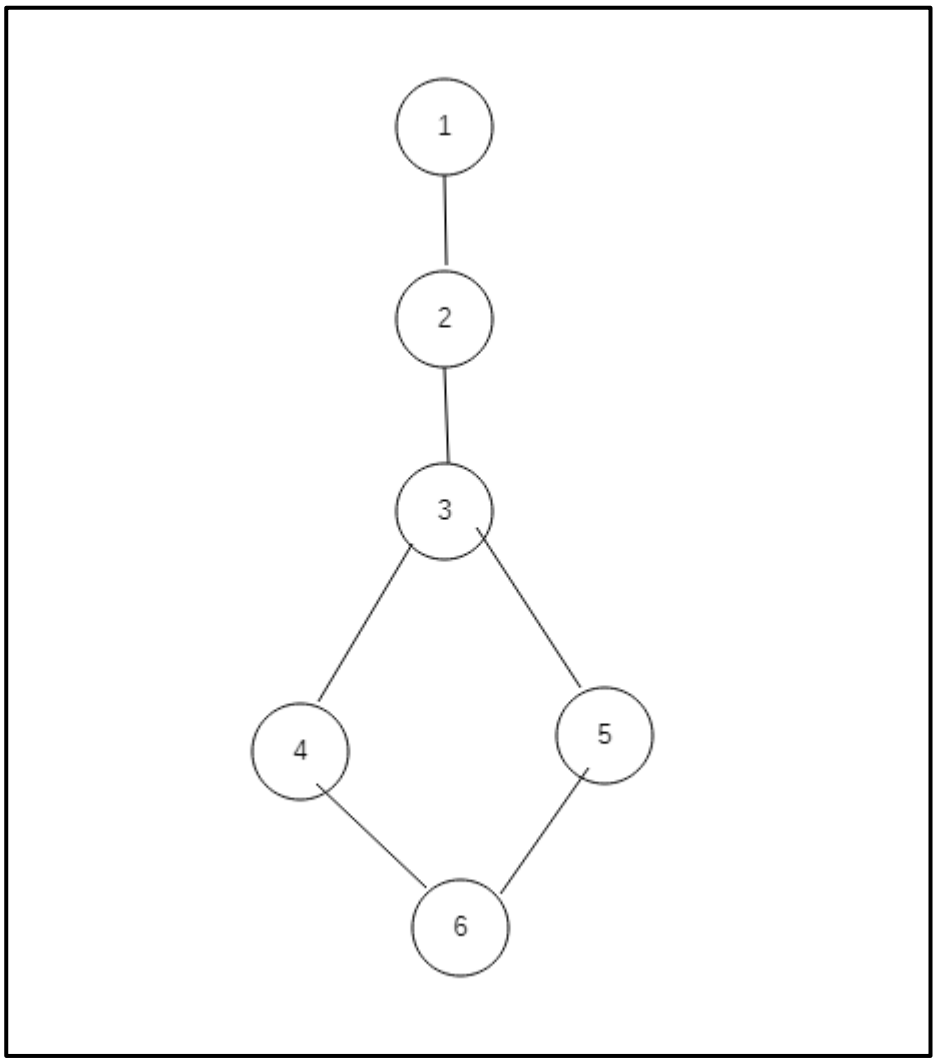
For the accident detection program:

Nodes represent:

Start

2 Input speed and impact

3 Check condition (speed > 60 and impact)
4 Accident detected
5 No accident
6 End



6.1.2 Cyclomatic Complexity (Graph Method)

Cyclomatic complexity is calculated using:
V(G) = E − N + 2
Where:
E = Number of edges
N = Number of nodes
Assume:
N = 6
E = 6
Therefore:
V(G) = 6 − 6 + 2
V(G) = **2**
This means **two independent execution paths exist**.

6.1.3 Independent Program Paths

Since **Cyclomatic Complexity = 2**, there are **2 independent paths**.

Path 1 – Accident Detected
1 → 2 → 3 → 4 → 6
Path 2 – No Accident
1 → 2 → 3 → 5 → 6

Independent Paths & Test Cases

Path No	**Independent Path**	**Test Input**	**Expected Output**
P1	1,2,3,4,6	Speed=80,impact=True	Accident Detected
P2	1,2,3,5,6	Speed=30,impact=Falsee	No Accident

6.1.4 Implement Unit Tests (Execution Phase)

Convert the above table into Python unittest.

```
import unittest

def detect_accident(speed, impact):
    If speed > 60 and impact == True:
        return True
    else:
        return False


class TestAccidentDetection(unittest.TestCase):

    def test_accident(self):
    self.assertTrue(detect_accident(80, True))

    def test_no_accident(self):
    self.assertFalse(detect_accident(30, False))


    if __name__ == "__main__":
        unittest.main()
```

Output:-

All tests passed
So, white-box testing is Successful

6.2 Black-Box Testing

Black-Box Testing verifies the **functional behaviour of the system without examining the internal code**.

This testing focuses on:

Inputs

Outputs

System behaviour

Sample Code-1 (Black Box)

Module-2: Accident Detection

```
def detect_accident(speed, impact):

    if speed > 60 and impact == True:
        return True
    else:
        return False
```

Step-1: Functional Requirements

From the accident detection module:

- System shall accept vehicle speed as input
- System shall detect sudden impact using sensors
- If speed > 60 and impact detected → accident occurs
- If conditions are not satisfied → no accident
- If accident occurs → emergency alert must be triggered

Step-2 Black Box Testing Techniques

2.1 Equivalence Partitioning

Inputs are divided into logical classes.

Speed Input

Class	Condition	Type
EC1	Speed<=0	Invalid
EC2	1<=Speed<=60	Valid

EC3	Speed> 60	Valid(Accident Possible)

Impact Input

Class	Condition	Type
EC4	Impact=True	Valid
EC5	Impact=False	Valid

Combined Test Cases

Class	Speed	Impact	Expected Result
EC6	80	True	Accident Detected
EC7	50	True	No Accident
EC8	80	False	No Accident
EC9	40	False	No Accident

2.2 Boundary Value Analysis

Testing is done on **boundary limits**.

Speed Boundary Values

BB No	Speed	Impact	Expected Output
BB1	**0**	**True**	**No Accident**
BB2	**1**	**False**	**No Accident**
BB3	**60**	**True**	**No Accident**
BB4	**61**	**True**	**Accident Detected**

Example Test Execution

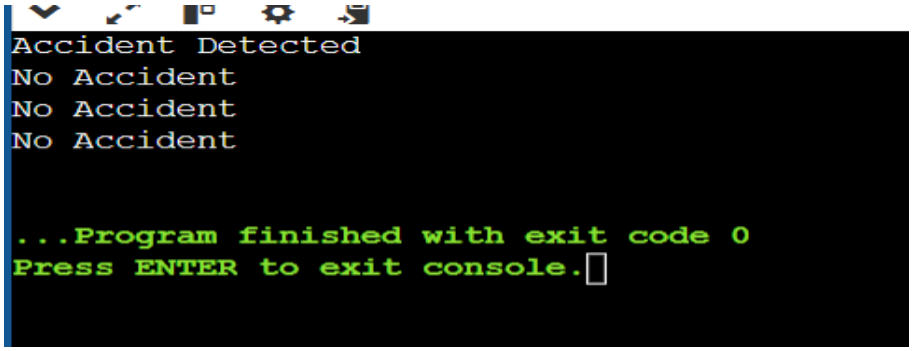
Test Cases	Speed	Impact	Result
TC1	70	True	Accident Detected
TC2	30	True	No Accident
TC3	80	False	No Accident
TC4	61	True	Accident Detected

Run the Program manually or using function calls.

Code:-

```
def detect_accident(speed, impact):

if speed > 60 and impact == True:
    return "Accident Detected"
else:
    return "No Accident"


# Test Inputs
print(detect_accident(80, True))    # Test Case 1
print(detect_accident(50, True))    # Test Case 2
print(detect_accident(80, False))   # Test Case 3
print(detect_accident(40, False))
```

Output:-



If outputs match expected results, **black-box testing is successful**.

6.1 White Box Testing

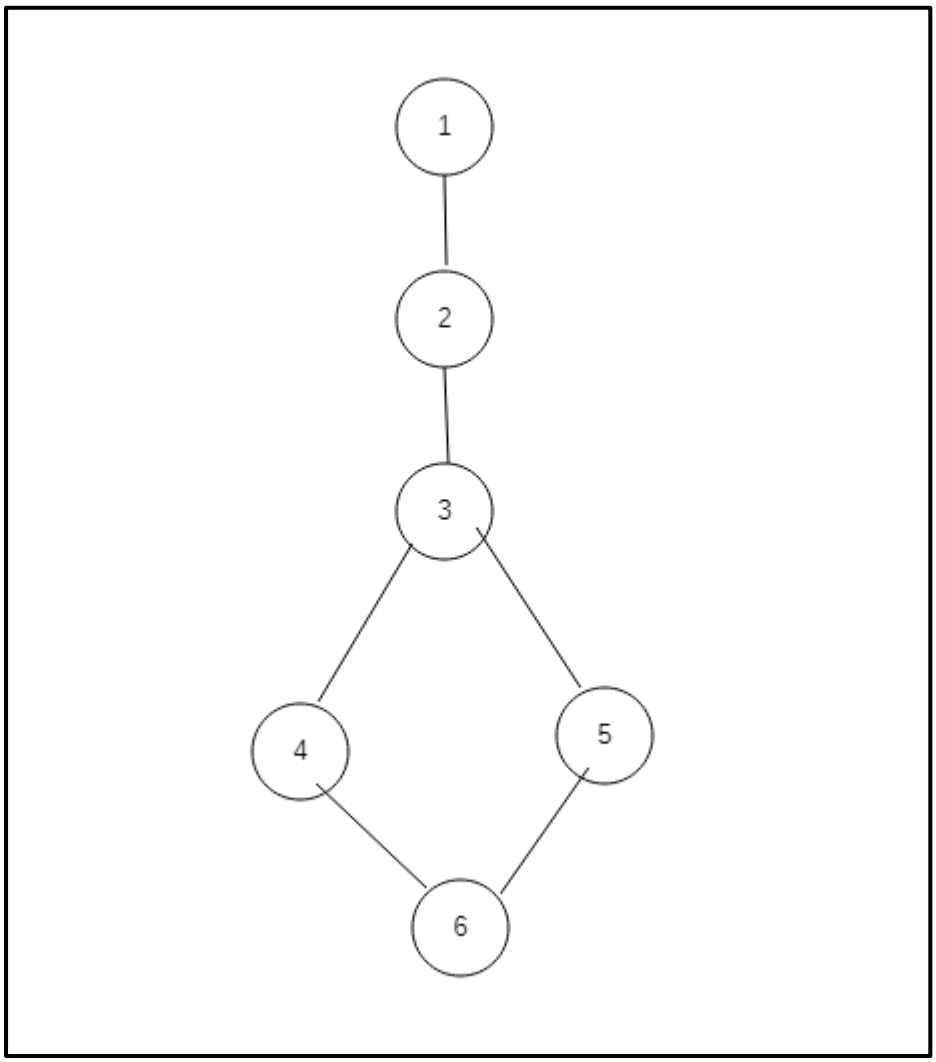
Sample Code-2

Module-3: Emergency Alert System

```
def send_alert(accident_detected, location):

    if accident_detected == True and location != "":
        return "Emergency Alert Sent"
    else:
        return "No Alert Sent"


  # Main Program

accident_detected = input("Accident detected (True/False): ")
location = input("Enter GPS Location: ")

result = send_alert(accident_detected, location)

print(result)
```

6.1.1 Flow Graph Notation

Control Flow Graph represents the logical flow of the program.
Nodes represent statements or decisions
Edges represent the flow of control between nodes



6.1.2 Cyclomatic Complexity (Graph Method)

Cyclomatic complexity is calculated using:
V(G) = E − N + 2
Where:
E = Number of edges
N = Number of nodes
Assume:
N = 6
E = 6
Therefore:

V(G) = 6 − 6 + 2
V(G) = 2
This means two independent execution paths exist.

6.1.3 Independent Program Paths

Since Cyclomatic Complexity = 2, there are 2 independent paths.

Path 1 – Alert Sent
1 → 2 → 3 → 4 → 6

Path 2 – Alert Not Sent
1 → 2 → 3 → 5 → 6

Independent Paths & Test Cases

Path No	Independent path	Test Input	Expected Output
P1	1,2,3,4,6	accident_detected = True, location = "Guntur"	Emergency Alert Sent
P2	1,2,3,5,6	accident_detected = False, location = ""	No Alert Sent

6.1.4 Implement Unit Tests (Execution Phase)

Code:-

```
import unittest

def send_alert(accident, location):
    if accident == True and location != "":
        return "Emergency Alert Sent"
    else:
        return "No Alert Sent"


class TestEmergencyAlert(unittest.TestCase):
```

```
    def test_alert(self):
        self.assertEqual(send_alert(True, "Guntur"), "Emergency Alert Sent")

    def test_no_alert(self):
        self.assertEqual(send_alert(False, ""), "No Alert Sent")


if __name__ == "__main__":
    unittest.main()
```

Output:-

If all test cases pass, the white-box testing is successful.

6.2 Black-Box Testing

Functional behavior verified without analyzing internal code

Sample Code-02 (Black Box)
Module-3: Emergency Alert System

```
def send_alert(accident_detected, location):

    if accident_detected == True and location != "":
        return "Emergency Alert Sent"
    else:
        return "No Alert Sent"
```

Step-1: Functional Requirements

From the emergency alert module:
System shall detect accident status
System shall obtain GPS location
If accident is detected and location available → emergency alert must be sent
If accident is not detected → no alert should be sent
System must notify emergency contacts

Step-2 Black Box Testing Techniques

2.1 Equivalence Partitioning

Inputs are divided into logical classes.

Accident Status Input

Class	**Condition**	**Type**
EC1	accident_detected = True	Valid
EC2	accident_detected = False	Valid

Location Input

Class	**Condition**	**Type**
EC3	Location available	Valid
EC4	Location empty	Invalid

Combined Test Cases

Class	**Accident**	**Location**	**Expected Result**
EC5	**True**	Available	Emergency Alert Sent
EC6	**False**	Available	No Alert Sent
EC7	**True**	Empty	No Alert Sent
EC8	**False**	Empty	No Alert Sent

2.2 Boundary Value Analysis

Testing is done on boundary limits.

Location Boundary Values

BB No	Accident	Location	Expected Output
BB1	False	""	No Alert Sent
BB2	True	""	No Alert Sent
BB3	True	"Location Available"	Emergency Alert Sent
BB4	False	"Location Available"	No Alert Sent

Example Test Execution

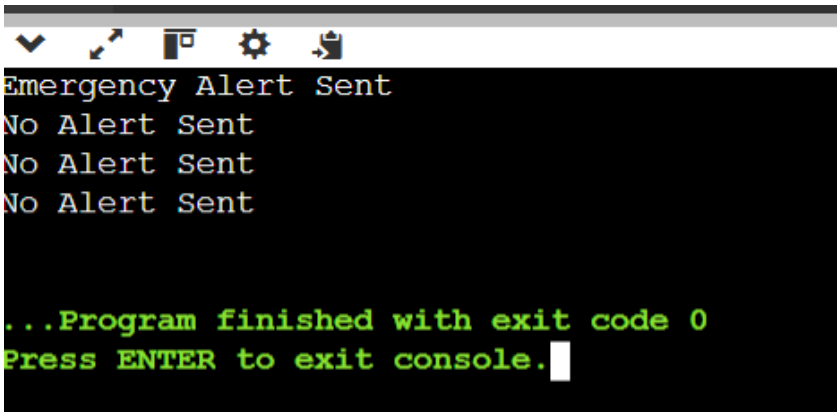
Test Case	Accident	Location	Result
TC1	True	Guntur	Emergency Alert Sent
TC2	False	Guntur	No Alert Sent
TC3	True	""	No Alert Sent
TC4	False	""	No Alert Sent

Run the Program manually or using function calls.

Code:-

```
def send_alert(accident_detected, location):

    if accident_detected == True and location != "":
        return "Emergency Alert Sent"
    else:
        return "No Alert Sent"


# Test Inputs for Black Box Testing
print(send_alert(True, "Guntur"))      # Test Case 1
print(send_alert(False, "Guntur"))     # Test Case 2
print(send_alert(True, ""))            # Test Case 3
print(send_alert(False, ""))           # Test Case 4
```

output:-



The Output is matched to all the above test cases mentioned in table, then Black Box testing is successful

7. Conclusion

The Software Requirement Specification (SRS) document for the Smart Accident Detection and Emergency Alert System provides a detailed description of the system's functional and non-functional requirements along with the overall system structure. The system is designed to improve road safety by automatically detecting accidents and sending emergency alerts to nearby contacts and hospitals using mobile sensors and GPS technology.

The application allows users to register and manage emergency contacts while the system continuously monitors sensor data during travel. When an accident is detected, the system obtains the user's location and sends alert notifications to emergency contacts and hospitals to ensure quick assistance.

This system helps reduce the delay in emergency response and provides a reliable solution for accident detection and notification. By clearly defining the requirements and system behavior, this SRS document acts as a foundation for system design, implementation, testing, and future development.

In the future, the system can be enhanced by integrating additional features such as improved accident detection algorithms, integration with government emergency services, and advanced security mechanisms to further improve the reliability and efficiency of the system.

8. References

- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- IEEE Computer Society. (2018). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830). IEEE.
- Object Management Group. (2017). Unified Modeling Language (UML) Specification. Retrieved from https://www.omg.org/uml
- Android Developers. (2024). Android Sensor and Location API Documentation. Retrieved from https://developer.android.com
- Diagrams.net. (2024). Draw.io Diagram Tool Documentation. Retrieved from https://www.diagrams.net

9. Acknowledgements

We would like to express our sincere gratitude to our faculty members and institution for giving us the opportunity to work on the Smart Accident Detection and Emergency Alert System project. Their guidance helped us understand important concepts related to software engineering, mobile application development, and system design.

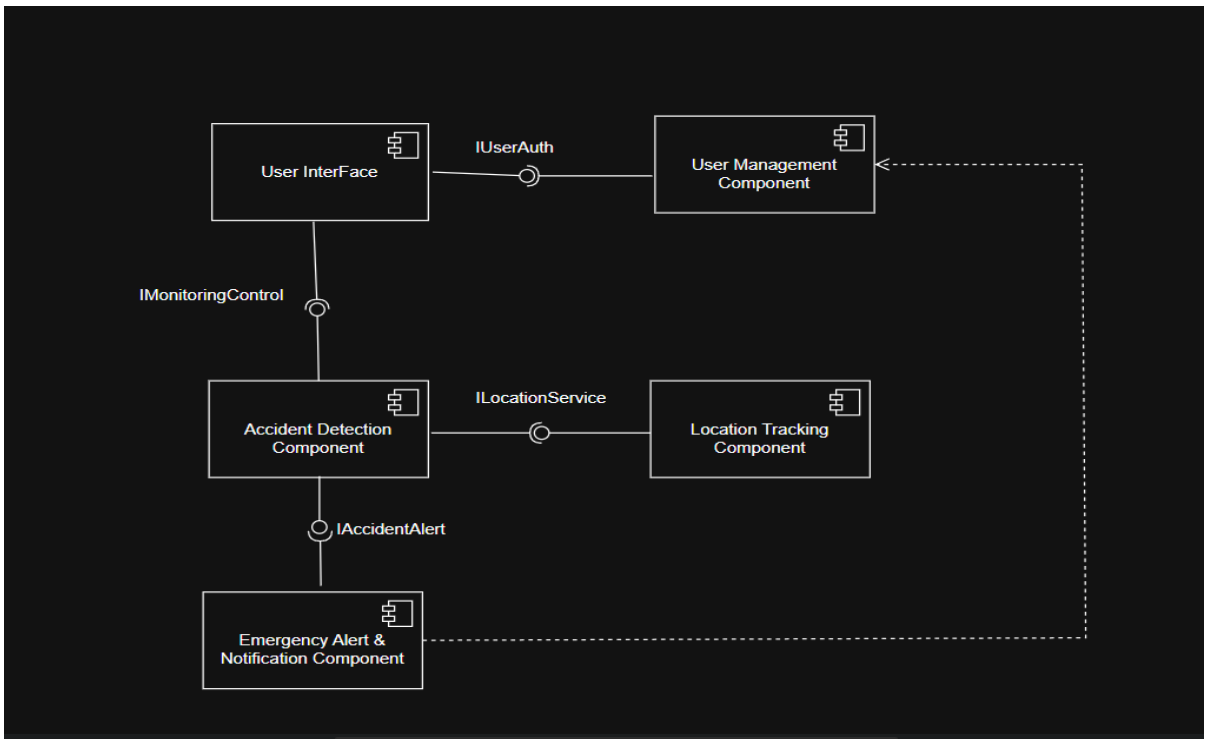
We would also like to thank our project guide for their valuable suggestions, encouragement, and continuous support throughout the development of this project. Their guidance helped us improve the quality of our work and successfully complete this Software Requirement Specification (SRS) document.
Finally, we extend our thanks to our classmates and friends for their support, cooperation, and helpful discussions during the preparation of this project documentation.

DESIGN ARCHITECTURE

The design phase of the Smart Accident Detectection And Emergency Alert System Using Andriod describes the structure and deployment of the system using UML diagrams.

10.1. Component Diagram:

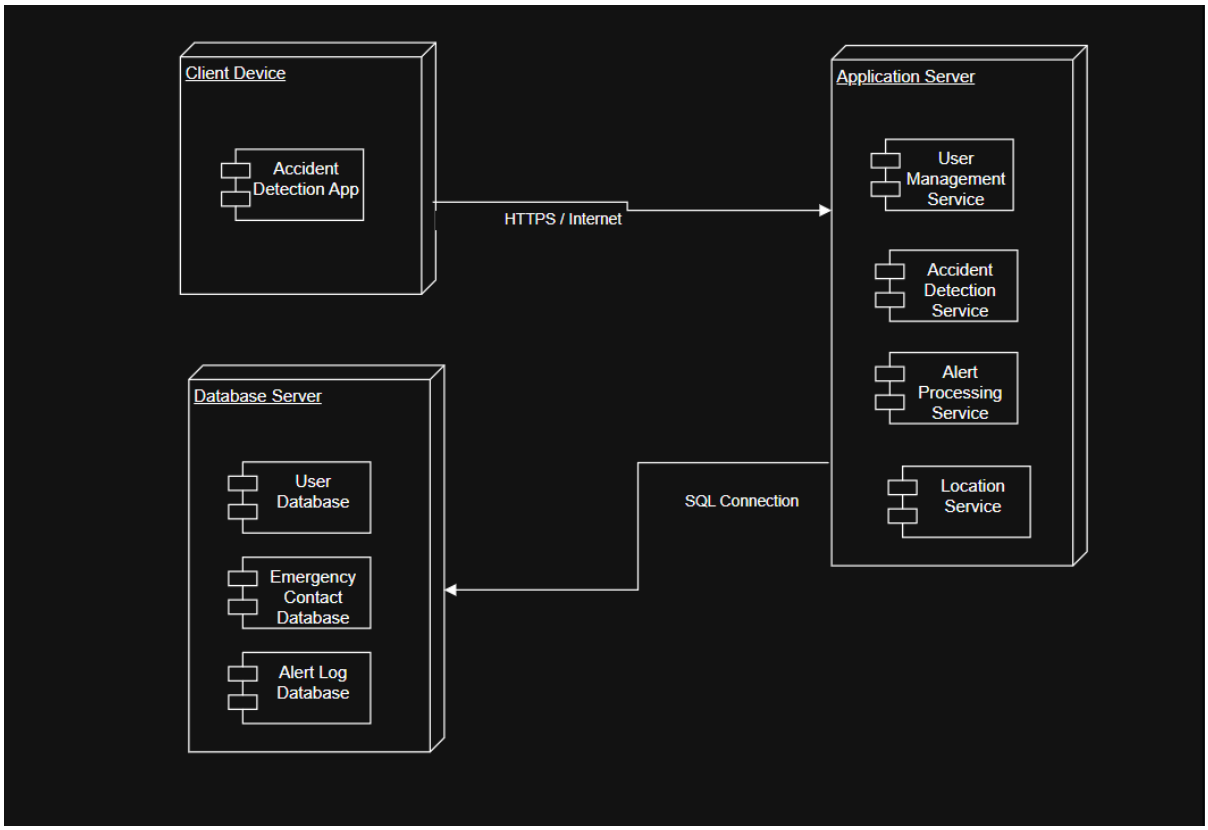
The component diagram represents the major software components of the Smart Accident Detection and Emergency Alert System and their interactions. It shows how components such as the User Interface, User Management, Accident Detection, Location Tracking, and Emergency Alert & Notification modules communicate with each other through defined interfaces and dependencies. These components work together to detect accidents, obtain location information, and send emergency alerts to contacts and hospitals.



10.2. Deployment Diagram:

The deployment diagram illustrates the physical architecture of the Smart Accident Detection and Emergency Alert System. It shows how the system is

deployed across different nodes such as the client device (Android smartphone), application server, and database server, and how these nodes communicate with each other through internet and database connections to process accident detection, location tracking, and emergency notifications.



PRODUCT METRICS FOR SYSTEM ANALYSIS MODEL

1.Product Metrics

- Product metrics measure the quality and functionality of the software product.
- For the Smart Accident Detection and Emergency Alert System, these metrics help evaluate:
- System functionality
- Complexity
- Maintainability
- Performance

2. Analysis Model Metrics

- Analysis model metrics estimate the size and complexity of the system during requirement analysis.
- For the Smart Accident Detection and Emergency Alert System, the main metrics include:

1) External Inputs (EIs): 5

- Inputs are data entered by the user or collected by the system.
- Examples:
- User registration details
- Login credentials
- Emergency contact information
- Sensor data from the mobile device
- Manual SOS alert activation

2) External Outputs (EOs): 5

- Outputs are responses generated by the system.
- Examples:
- Accident detection alert
- Emergency notification to contacts
- Location sharing message
- SOS alert message

- Hospital emergency notification

3) External Inquiries (EQs): 4

- User inquiries are requests where users retrieve information without modifying data.
- Examples:
- View user profile
- View emergency contact list
- Check accident alert status
- View location details

4) Internal Logical Files (ILFs): 4

- ILFs are data stored and maintained by the system.
- Examples:
- User database
- Emergency contact database
- Accident log database
- Alert history database

5) External Interface Files (EIFs): 2

- EIFs are data used by the system but maintained by another system.
- Examples:
- GPS location service
- Hospital information system

3.Function Point Calculation

Step 1: Calculate UFP(Unadjusted Function Points)

Component	Count	Weight	Total
Inputs(EI)	5	4	5*4=20
Outputs(Eo)	5	5	5*5=25
Queries(EQ)	4	4	4*4=16

Logical Files	4	10	4*10=40
Interface Files	2	7	2*7=14

Total UFP=20+25+16+40+14=115

UFP=115

4. Complexity Adjustment Factor (CAF)

- CAF formula:
- CAF = 0.65 + (0.01 × ΣFi)
- Where:
- is the scaling factor
- 0.65 is the base adjustment value

Value Adjustment Factors

- Reliable backup and recovery required? – Score: 4 – Accident logs and user data must be securely stored.
- Specialized data communication required? – Score: 4 – The system communicates using internet services and GPS.
- Distributed processing functions present? – Score: 3 – Processing occurs between mobile device and server.
- Is performance critical? – Score: 5 – Accident alerts must be sent immediately.
- Runs in heavily used environment? – Score: 3 – Many users may use the application simultaneously.
- Online data entry required? – Score: 4 – Users register and add emergency contacts.
- Multi-screen online data entry required? – Score: 3 – Registration and contact management involve multiple screens.
- ILFs updated online? – Score: 4 – User and accident data are updated in real time.
- Inputs/outputs/files complex? – Score: 3 – Sensor data and location information add moderate complexity.

- Internal processing complex? – Score: 4 – Accident detection using sensor data requires processing.
- Code designed for reuse? – Score: 3 – Modules like authentication and notifications can be reused.
- Conversion and installation included? – Score: 2 – Application installation is simple.
- Multiple installations required? – Score: 3 – System can be used by many users and hospitals.
- Designed for change and ease of use? – Score: 4 – System supports user-friendly interface and future updates.

- ΣFi Calculation
- ΣFi =4 + 4 + 3 + 5 + 3 + 4 + 3 + 4 + 3 + 4 + 3 + 2 + 3 + 4
- ΣFi = 49
- CAF Calculation
- CAF = 0.65 + (0.01 × 49)
- CAF = 0.65 + 0.49
- CAF = 1.14

5. Final Function Point

FP = UFP × CAF
FP = 115 × 1.14
FP = 131.1

Therefore,
Function Point (FP) = 131.1

What 131.1 Function Points Means

The value 131.1 Function Points represents the functional size of the Smart Accident Detection and Emergency Alert System after adjusting for system complexity.
It indicates that the system is a moderately complex software application with features such as accident detection, location tracking, emergency alert

generation, user management, and communication with emergency contacts and hospitals.

Development Effort Estimation

1 Function Point = 10 development hours
131.1 × 10 = 1311 hours

Estimated development effort:

1311 hours

Project Duration Based on Number of Developers

Number of Developers	Estimated Work Hours
1 Developer	1311 hours
2 Developers	656 hours
3 Developers	437 hours
4 Developers	328 hours

Example:
If 4 developers work 6 hours per day
328 ÷ 6 = 54.6 ≈ 55 days
Estimated project duration ≈ 8 weeks

Productivity Measurement

Productivity = FP / Development Hours
Productivity = 131.1 / 1311
Productivity = 0.1 FP/hour

Possible Number of Errors

Industry average:
Errors per Function Point = 0.5 – 1
Estimated errors:
131.1 × 0.7 = 91.77 ≈ 92 errors
Estimated errors = 92

Error Fixing Effort

Average time to fix one error = 3 hours
92 × 3 = 276 hours
Error fixing effort = 276 hours

Fixing Cost

Developer cost = ₹1000 per hour
276 × 1000 = ₹2,76,000
Error fixing cost = ₹2,76,000

Testing Effort

Testing requires 30% – 40% of development effort
Taking 35%
1311 × 0.35 = 458.85 ≈ 459 hours
Testing effort = 459 hours

Testing Cost

459 × 1000 = ₹4,59,000

Testing cost = ₹4,59,000

Total Project Cost

Development Cost = ₹13,11,000
Error Fixing Cost = ₹2,76,000
Testing Cost = ₹4,59,000

Total Cost:

13,11,000 + 2,76,000 + 4,59,000 = ₹20,46,000

Final Estimated Project Cost
Total Estimated Project Cost = ₹20,46,000

EMPIRICAL ESTIMATION MODEL (COCOMO MODEL)

COCOMO MODEL:

The COCOMO (Constructive Cost Model) is a standard method used in software engineering to estimate the effort, cost, and schedule required for a software project. Introduced by Barry Boehm in 1981, it relies on historical data and mathematical formulas to predict project outcomes based primarily on the size of the software

There are 2 Types of Models:

1. Basic COCOMO Model
2. Intermediate COCOMO Model

1.Basic COCOMO Model:

The Basic COCOMO can be used for quick and slightly rough calculations of Software Costs.

The following formulas are used to calculate effort and time estimates:

- Effort (E) = A*(KLOC)^B
- Time (T) = C*(Effort)^D
- Persons required = Effort / Time

The constant values A, B, C and D for the Basic Model for the different categories of software projects.

Software project	A	B	C	D
Organic	2.4	1.05	2.5	0.38
Semi Detached	3.6	1.12	2.5	0.35
Emmbeded	3.0	1.20	2.5	0.32

The Smart Accident Detection And Emergency Alert System is categorized under the semi-detached, With a Size of 6.55KLOC.

A=3.6
B=1.12
C=2.5
D=0.35

- **Effort = 3.0× (6.55)^1.12 = 3.0 × 8.1 ≈ 24.3 person-months**
- **Time = 2.5 × (24.3)^0.35 ≈ 2.5 × 3.1 ≈ 7.75 months**
- **Persons required = 24.3 / 7.75 ≈ 3 persons**

Basic COCOMO result of Smart Accident Detection And Emergency Alert System:

- **KLOC = 6.55**
- **Effort = 24.3 PM**
- **Time = 7.75 months**
- **Team size = 3 persons**

2.Intermediate COCOMO Model:

The Intermediate COCOMO model estimates software effort by considering both size (KLOC) and cost drivers. It uses an Effort Adjustment Factor (EAF) to account for factors like product complexity, team capability, and tools, making it more accurate than the basic model

- In the intermediate COCOMO model, various other factors such as reliability, experience, and Capability are considered along with LOC to estimate effort and time schedule.
- These factors are known as Cost Drivers and the Intermediate Model utilizes 15 such drivers for cost estimation

Product attributes

1. Required software reliability

Since payment and registration data should be correct, reliability matters.
Rating: High = 1.15

2. Size of application database

Database is there, but not huge like banking or hospital systems. Rating:
Nominal = 1.00 3.

3. Product complexity

Your project has login, event handling, registration, payment, email, and DB integration, so complexity is above normal.
Rating: High = 1.15

Hardware attributes

4.Run-time performance constraints

No strict real-time or high-performance requirement.
Rating: Nominal = 1.00

5.Memory constraints

No serious memory limitation.
Rating: Nominal = 1.00

6.Volatility of virtual machine environment

Normal development environment, not highly unstable.
Rating: Nominal = 1.00

7.Required turnabout time

No urgent machine turnaround requirement.
Rating: Nominal = 1.

Personnel attributes

8. Analyst capability

For a student project, we can keep this at average.
Rating: Nominal = 1.00

9. Software engineering capability

Again, average student-team assumption.
Rating: Nominal = 1.00

10. Application experience

You are building this type of project for learning, so domain experience is not very high.
Rating: Low = 1.17

11. Virtual machine

experience Normal experience assumed.
Rating: Nominal = 1.00

12. Programming language experience

You are doing it in Java and have already been working on it, so average is fair.
Rating: Nominal = 1.0

Project attributes

13. Use of software tools

You are using development tools, DB tools, diagram tools, etc., so tool usage is good**.**

Rating: High = 0.91

14. Application of software engineering methods

Since you are preparing SRS, UML, testing, DFD, ERD, and design properly, methods are above normal.

Rating: High = 0.91

15. Required development schedule

No extreme deadline pressure assumed.

Rating: Nominal = 1.00

Tabular Representation

Cost Drivers	Very Low	Low	Nominal	High	Very High	Extra High
1					1.15	
2				1.00		
3					1.15	
4				1.00		
5				1.00		
6				1.00		
7				1.00		
8				1.00		
9				1.00		
10		1.17				
11				1.00		
12				1.00		
13					0.91	
14					0.91	
15				1.00		

EAF= Product of cost drivers

EAF = 1.15 × 1.00 × 1.15 × 1.00 × 1.00 × 1.00 × 1.00 × 1.00 × 1.00 × 1.17 × 1.00 × 1.00 × 0.91 × 0.91 × 1.00

EAF = 1.2813 ≈ 1.28

The following formulas are used to calculate effort and time estimates:

- Effort (E) = A*(KLOC)^B*(EAF)
- Time (T) = C*(Effort)^D
- Persons required = Effort / Time

The Smart Accident Detection And Emergency Alert System is categorized under the semi-detached, With a Size of 6.55KLOC.

A=3.6
B=1.12
C=2.5
D=0.35

Effort

Effort = 3.2 × (6.55)^1.12 × 1.28

Effort ≈ 31.1 person-months

Development time

Time = 2.5 × (31.1)^0.35

Time ≈ 8.25 months

Team size

Persons required = 31.1 / 8.25 ≈ 4 persons